Training Copilot: A Multi-Agent Sports Analytics Platform Using the Model Context Protocol

Seminar Paper
AI in Service Systems II

Group 6

Maximilian Klasen, Aaron Neumann, Lorenz Neugebauer,
Marvin Janzen, Simon Meyer

At the Department of Economics and Management

Digital Service Innovation (DSI)

Karlsruhe Digital Service Research & Innovation Hub (KSRI) &
Institute for Information Systems (WIN)

Examiner:	Prof. Dr. Gerhard Satzger
Second Examiner:	Prof. Dr. Hansjörg Fromm
Date of Submission:	August 17, 2026

Declaration of Academic Integrity

We hereby confirm that the present work is solely our own work and that if any text passages or diagrams from books, papers, the Web or other sources have been copied or in any other way used, all references—including those found in electronic media—have been acknowledged and fully cited.

Karlsruhe, August 17, 2026

Maximilian Klasen, Aaron Neumann, Lorenz Neugebauer, Marvin Janzen, Simon Meyer

Abstract

Recreational and competitive athletes rely on multiple digital platforms to track training load, monitor recovery, plan routes, and schedule sessions. However, this data remains fragmented across proprietary ecosystems—Strava for activities, Garmin for wellness, weather services for planning, and calendars for availability—forcing users into manual cross-referencing that is time-consuming and error-prone. This paper presents *Training Copilot*, an open-source, multi-agent sports analytics platform that unifies these heterogeneous data sources behind a single conversational interface. Training Copilot adopts the Model Context Protocol (MCP) as a uniform integration layer: each external API is encapsulated in an independent MCP server, and a central host discovers and routes tool calls without hardcoding provider-specific logic. On top of this data layer, a LangGraph-based multi-agent system coordinates five specialist agents—recovery, training load, environmental context, route planning, and fitness knowledge—via the Agent-to-Agent (A2A) protocol. Each specialist is an autonomous Reasoning-and-Acting (ReAct) agent scoped to its own MCP servers. An orchestrator agent decomposes user queries, delegates to the appropriate specialists in parallel, and synthesises data-driven recommendations. The fitness knowledge specialist employs Retrieval-Augmented Generation (RAG) over a curated vector database of public-domain exercise science literature, grounding its advice in verifiable sources. We evaluate the system through an automated end-to-end assessment using MLflow's conversation simulation framework, employing ten synthetic personas across two user archetypes that collectively exercise all five specialist domains. Each conversation is scored along five dimensions—conversation completeness (whether the user's goal was achieved), user frustration (resilience under adversarial follow-ups), safety (absence of harmful advice), supportive coaching tone (empathy under pushback), and data grounding—where the last scorer is fully deterministic, verifying tool usage from execution traces rather than from chat text. The MCP-based architecture enables seamless extensibility—adding a new data source requires exactly one file and one configuration line—while the multi-agent design produces contextually aware, data-grounded training recommendations that integrate recovery status, workload trends, weather conditions, and route options into a single coherent response.

Contents

Acronyms

A2A	Agent-to-Agent (protocol)
ACWR	Acute-to-Chronic Workload Ratio
API	Application Programming Interface
ATL	Acute Training Load
BMI	Body Mass Index
CAGR	Compound Annual Growth Rate
CoT	Chain-of-Thought
CTL	Chronic Training Load
EMA	Exponential Moving Average
GPS	Global Positioning System
HR	Heart Rate
HRV	Heart Rate Variability
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MCP	Model Context Protocol
MFA	Multi-Factor Authentication
NLP	Natural Language Processing
OAuth	Open Authorization
ORS	OpenRouteService
OSM	OpenStreetMap
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
REST	Representational State Transfer
RPC	Remote Procedure Call
SBERT	Sentence-BERT
SpO_2	Peripheral Oxygen Saturation
SSE	Server-Sent Events
SSRF	Server-Side Request Forgery
TRIMP	Training Impulse
TSB	Training Stress Balance
TSS	Training Stress Score
UI	User Interface
UV	Ultraviolet
VO_2max	Maximal Oxygen Uptake

List of Figures

List of Tables

1 Introduction

Wearable devices from Garmin, Polar, and Apple now capture heart rate, GPS trajectories, sleep stages, heart-rate variability (HRV), and stress levels continuously (Seckin, Ates, & Seckin, 2023). Strava alone aggregates activity data from over 180 million users (Strava, 2025). The result is an unprecedented volume of training data—yet it remains fragmented across proprietary ecosystems with incompatible interfaces, making holistic analysis impractical for anyone without a sports-science background (Bourdon et al., 2017).

A concrete example illustrates this. An endurance cyclist we encountered during user research exports his Strava data after every ride, uploads it to ChatGPT, and asks the model to analyse his progress. The process is both slow and incomplete: the general-purpose LLM has no access to his Garmin recovery data, the weather forecast, or his calendar. More broadly, deciding whether to train on a given day requires checking a Garmin watch for HRV and sleep quality, Strava for cumulative load, a weather service for conditions, and a calendar for scheduling. Each tool requires a separate application, and the integration is left entirely to the athlete.

Large language models have demonstrated the ability to reason across heterogeneous information sources when equipped with tool-calling capabilities (Qu et al., 2025; Schick et al., 2023). The emergence of open interoperability protocols—Anthropic's Model Context Protocol (MCP) (Anthropic, 2024b) for tool discovery and invocation, and Google's Agent-to-Agent (A2A) (Google Cloud, 2025) for inter-agent coordination—makes it feasible to build a system that connects these fragmented sources behind a single conversational interface.

1.1 Problem Statement and Approach

This paper presents *Training Copilot*, an open-source sports analytics platform that addresses three problems:

Data fragmentation. Training and wellness data is distributed across platforms with incompatible authentication models, data formats, and access patterns. Sports science consensus statements emphasise that effective load monitoring requires combining external and internal load from multiple sources (Bourdon et al., 2017; Halson, 2014), yet the consumer tools that collect this data do not provide a unified view. Training Copilot solves this with MCP as a uniform data layer: each external API is encapsulated in an independent MCP server, and a single host client discovers and routes tool calls without hardcoding provider-specific logic.

Lack of cross-domain reasoning. The platforms we examined—Strava, Garmin Connect, Intervals.icu, TrainingPeaks—display metrics in isolation: a sleep score here, a training load chart there. An athlete's readiness to train depends on the interplay of recovery status, cumulative workload, environmental conditions, and personal schedule (Bourdon et al., 2017; Gabbett, 2016), but none of these tools synthesise across all four dimensions. Training Copilot addresses this with a multi-agent system: five LangGraph (LangChain, 2024) ReAct (Yao et al., 2023) agents, each scoped to a specific domain (recovery, load, context, routes, fitness knowledge), coordinated by an orchestrator via the A2A protocol.

Rigid architectures. The platforms listed above are tightly coupled to their data sources—adding a new source (e.g., a nutrition tracker) would require code changes throughout their stack. Training Copilot's MCP-based design reduces adding a new data source to one server file and one configuration line, consistent with microservice design principles (Dragoni et al., 2017).

1.2 Contributions

- An MCP-based integration architecture that achieves single-line extensibility for new data sources, verified empirically during development by adding the Telegram bridge and flythrough servers without changes to the host, agents, or UI.
- A multi-agent system with five domain-scoped specialists coordinated via A2A, where scoping each specialist to at most two MCP servers addresses the tool-selection degradation observed with large tool sets (Qu et al., 2025).
- A RAG-based fitness knowledge agent that grounds exercise-science advice in a local vector database of public-domain literature, with explicit citation preservation through a post-processing step.
- An automated evaluation framework using MLflow conversation simulation (MLflow, 2026) with ten persona-driven test cases and five scoring dimensions, including a deterministic trace-based grounding scorer.

1.3 Structure of This Paper

Section 2 analyses the sports analytics market and identifies the gap Training Copilot addresses. Section 3 reviews the state of the art in agentic AI, tool integration protocols, sports analytics, and retrieval-augmented generation. Section 4 presents the system architecture. Section 5 describes the data sources and their integration. Section 6 details the agent interaction patterns. Section 7 describes the evaluation methodology. Section 8 discusses trade-offs, limitations, and ethical considerations. Section 9 concludes with a summary and directions for future work.

2 Market Analysis

This section examines the sports analytics and fitness technology market, identifies the competitive landscape, and articulates the specific gap that Training Copilot addresses. We conclude with a personal perspective on what motivated our work.

2.1 Market Size and Growth

The global sports analytics market has experienced substantial growth, driven by the convergence of wearable technology adoption, cloud computing, and the increasing availability of large language models for consumer applications. Grand View Research estimates the global sports analytics market at USD 5.7 billion in 2025, projecting it to reach USD 23.1 billion by 2033 at a compound annual growth rate (CAGR) of 18.5 % (Grand View Research, 2026). The adjacent wearable fitness technology market continues to expand rapidly (IDC, 2024), indicating a massive and growing install base of data-generating devices.

The demand is particularly pronounced in the German market, where fitness app downloads have risen from 41.37 million in 2017 to a projected 135.22 million in 2025 (Statista, 2025). Strava's 2025 trend report, based on a survey of 30,494 respondents drawn from both its 180-million-user community and a random sample of non-users, reveals that 30 % of Generation Z plan to increase their fitness spending in 2026 and that 46 % of all respondents would use AI as a smart coach for sports—with Generation Z embracing AI-driven workout insights at a higher rate than older cohorts, though the report does not break out a separate Gen Z percentage for this question (Strava, 2025). A parallel trend is the rapid expansion of the generative AI market. McKinsey estimates that generative AI could contribute between USD 2.6 trillion and USD 4.4 trillion annually across industries, with particular impact in knowledge-intensive domains where synthesis of heterogeneous information is the primary task (McKinsey & Company, 2023). Sports coaching and wellness—where the end user must integrate physiological data, environmental conditions, and scheduling constraints—is precisely such a domain.

2.2 Competitive Landscape

Table 1 summarises the major platforms that athletes currently use to manage training data, along with their key limitations.

A recent wave of AI-powered coaching features has emerged within existing wearable ecosystems. WHOOP launched WHOOP Coach (WHOOP, 2024), an OpenAI-powered conversational assistant that answers natural-language questions about the

Table 1 Competitive landscape of consumer sports analytics platforms. No existing platform simultaneously provides multi-source integration, cross-domain reasoning, and conversational interaction.

Platform	Core Capability	Data Sources	Key Limitation
Strava	Activity tracking, social, routes	Strava uploads	No recovery data, no weather, no cross-source reasoning
Garmin Connect	Wellness, sleep, HRV, Body Battery	Garmin only	Walled garden; no Strava load, no route planning
TrainingPeaks	Training plans, TSS/CTL/ATL	File import	Paid, no chat interface, manual import
Intervals.icu	Load analytics, free tier	Strava, Garmin	No weather, calendar, or routes; dashboard-only
WHOOP Coach	AI chat on biometrics	WHOOP device	Walled garden; no Strava, routes, or calendar
Athletica.ai	Adaptive training plans	Garmin, Strava	Plan-only; no recovery chat, no route planning
ChatGPT / Gemini	General-purpose LLM	None (no API)	No live fitness data; generic, ungrounded advice

user's biometric data—but only WHOOP data, with no access to training load from Strava, weather forecasts, or calendar context. Adaptive plan generators such as Athletica.ai (Athletica.ai, 2024) dynamically adjust endurance training plans based on Garmin and Strava data, but operate as plan-output systems without a conversational interface or cross-domain reasoning. These platforms represent meaningful progress, but each remains siloed within its own data ecosystem and offers either a conversational interface *or* adaptive planning, never both grounded in multi-source data simultaneously.

The competitive landscape can also be understood along two axes: the breadth of data integration and the depth of AI-driven analysis (Figure 1). Point solutions (Garmin, Strava, weather apps) each cover a single data domain. Aggregation dashboards (Intervals.icu, TrainingPeaks) combine data from multiple sources but provide no agentic reasoning. AI-based training apps (Freeletics, various start-ups) apply machine learning but typically lack access to the user's actual wearable data. Training Copilot targets the upper-right quadrant: broad data integration combined with deep, agentic AI reasoning.

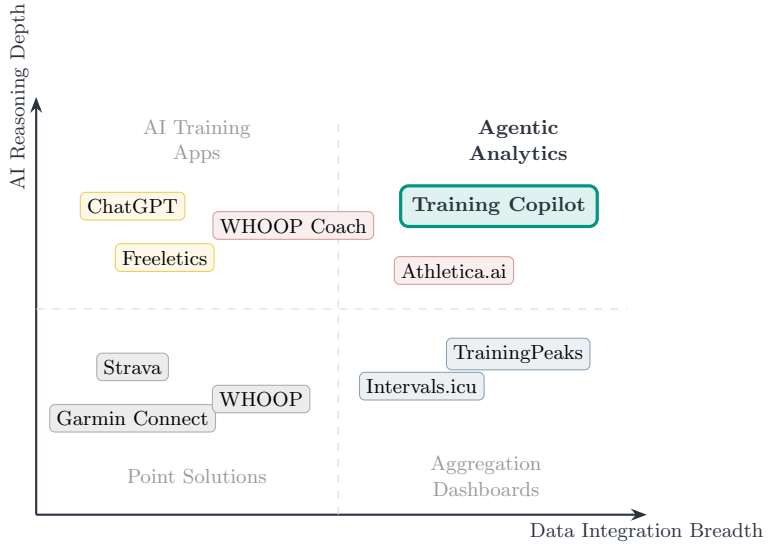

Fig. 1 Solution landscape. Point solutions cover single data domains; aggregation dashboards combine data but lack reasoning; AI training apps reason but lack data access. Training Copilot combines broad integration with deep agentic reasoning.

Several observations emerge from this landscape. First, no platform integrates all five data dimensions that matter for training decisions: activity history, wellness metrics, weather, calendar availability, and route planning. Second, the platforms that do offer analytical depth (TrainingPeaks, Intervals.icu) operate purely as dashboards—they visualise data but do not reason across sources or offer natural-language interaction. Third, general-purpose LLMs such as ChatGPT can reason and converse, but they have no access to the athlete's actual data, producing generic advice that ignores individual context (Puce, Bragazzi, Currà, & Trompetto, 2025).

2.3 The Market Gap

We identify a clear gap at the intersection of three capabilities that no existing consumer product occupies simultaneously (Figure 2):

1. **Multi-source data integration** — the ability to unify data from wearables (Garmin), activity platforms (Strava), environmental services (weather), productivity tools (Google Calendar), and geographic services (OpenRouteService) through a single interface.

2. **Contextual, cross-domain reasoning** — the capability to synthesise signals from recovery status, training load trends, weather forecasts, and schedule availability into a single, coherent recommendation, rather than presenting isolated metrics (Bourdon et al., 2017).
3. **Conversational interaction** — a natural-language interface that allows athletes to ask complex, multi-faceted questions and receive answers grounded in their own live data, without navigating multiple dashboards or interpreting raw numbers.

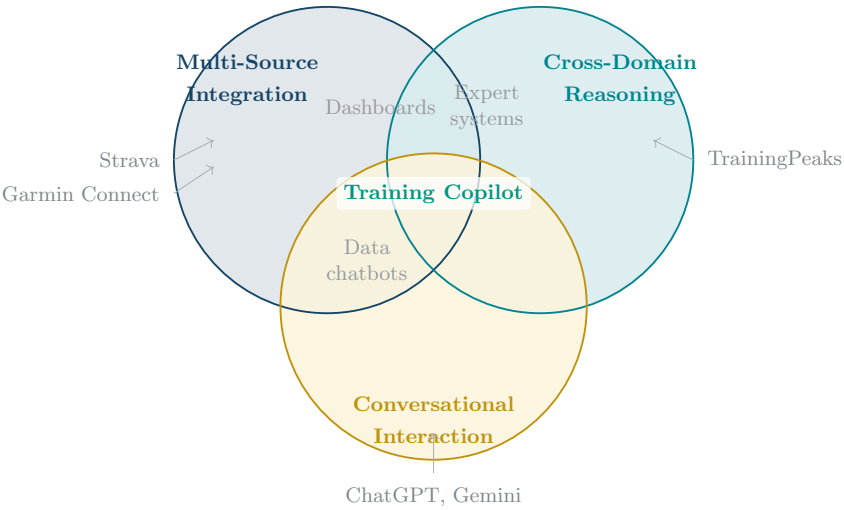

Fig. 2 The Training Copilot market gap. Existing platforms cover at most two of the three capabilities. Training Copilot is positioned at the intersection of all three, combining multi-source data integration (MCP layer), cross-domain contextual reasoning (multi-agent system), and conversational interaction (chat interface).

2.4 Target Users

Based on the market analysis, we identify two primary user archetypes, which also inform our evaluation design (Section 7):

The ambitious endurance athlete trains in structured blocks, tracks Training Stress Balance obsessively, wears a Garmin watch for HRV and recovery metrics, and wants precise, data-driven go/no-go decisions for key sessions. This user is frustrated by the need to cross-reference Strava trends with Garmin recovery data manually and by the fact that no platform integrates weather and calendar into training recommendations.

The recreational cyclist rides for enjoyment, trains by feel and social motivation, and cares about scenic routes, weather comfort, and easy planning. This user finds traditional analytics dashboards overwhelming and wants a simple, encouraging interface that provides practical answers ("Is tomorrow good for a ride?", "Find me a nice 40 km loop") without jargon.

These archetypes represent opposite ends of the analytical-depth spectrum, ensuring that Training Copilot must serve both data-hungry power users and casual athletes who prefer plain-language guidance.

2.5 Personal Motivation

The following reflects the author's personal perspective rather than an objective market assessment.

The motivation for Training Copilot emerged from direct frustration as an endurance athlete. Answering "Should I do my long run today or push it to Thursday?" meant checking five applications—Garmin for HRV and sleep, Strava for load trends, a weather service, Google Calendar, and sometimes a route planner—and mentally synthesising conflicting signals. The platforms that had the data offered no reasoning; the tools that could reason (ChatGPT, Gemini) had no data access. The emergence of MCP and A2A made it feasible to build the system we wished existed: one that treats this as a single question rather than five separate lookups.

3 State of the Art

This section reviews the current state of research across the four pillars that Training Copilot builds upon: agentic AI and multi-agent systems, tool integration protocols, sports analytics and wearable technology, and Retrieval-Augmented Generation.

3.1 Agentic AI and Multi-Agent Systems

The rapid scaling of large language models has enabled a paradigm shift from static text generation to autonomous *agentic* systems that plan, reason, and interact with external environments. The Transformer architecture (Vaswani et al., 2017), chain-of-thought prompting (Wei et al., 2022), and the scaling of model capacity to hundreds of billions of parameters (OpenAI, 2023) laid the groundwork by demonstrating that LLMs can perform multi-step reasoning when prompted to generate intermediate steps. We organise the subsequent literature along three themes: reasoning architectures, agent-internal structure, and multi-agent coordination.

3.1.1 Reasoning Architectures

Two complementary patterns have emerged for grounding LLM reasoning in external action. Yao et al. (2023) introduced *ReAct*, which interleaves reasoning traces ("thought" steps) and task-specific actions in a single LLM loop: the model generates a thought explaining its plan, executes an action (e.g., `search[entity]` against a Wikipedia API), observes the result, and repeats until the task is complete. Evaluated on PaLM-540B across four benchmarks, ReAct achieved 71 % success rate on ALFWorld (vs. 45 % for action-only) and 40 % on WebShop (vs. 30.1 % for action-only). On knowledge tasks (HotpotQA, FEVER), ReAct produced zero hallucinated reasoning chains, compared to 56 % for chain-of-thought prompting—because every factual claim is grounded in a retrieved observation rather than generated from the model's parameters. Training Copilot's specialist agents use this pattern directly: each is a LangGraph ReAct agent where the "actions" are MCP tool calls.

Shinn et al. (2023) extended this loop across episodes with *Reflexion*, where agents store linguistic self-reflections after failed attempts in an episodic memory buffer, informing subsequent trials without weight updates. The two approaches are complementary: ReAct provides the within-episode reasoning loop, while Reflexion adds cross-episode learning. Training Copilot uses Reflexion in one specific subsystem: the LLM-driven chart generator retries once with the error message as feedback when

execution fails (Section 4.7). Both patterns, however, assume a *single* agent operating over a *fixed* tool set—neither addresses how to coordinate multiple agents or how to discover tools dynamically from heterogeneous sources.

3.1.2 Agent-Internal Architecture

Several frameworks have sought to characterise the internal structure that makes an agent effective. Park et al. (2023) demonstrated that LLM-powered agents exhibit believable behaviour when equipped with three components: a *memory stream* recording experiences, a *reflection* mechanism synthesising abstractions, and a *planning* module translating reflections into action sequences. Wang et al. (2024) generalised these ideas into a four-module taxonomy—profiling, memory, planning, and action—that subsumes prior architectures. A common finding across both works is that memory and planning are mutually reinforcing: richer memory enables better planning, which in turn generates more informative experiences. However, both frameworks describe single-agent architectures; they do not address how multiple agents with different profiles and tool sets should coordinate.

3.1.3 Multi-Agent Coordination

The transition to multi-agent systems introduces coordination challenges absent from single-agent designs. Three approaches illustrate the design space. Shen et al. (2023) proposed HuggingGPT, where a central controller decomposes requests, selects specialist models from a hub, executes them, and summarises results—demonstrating the viability of orchestrator–specialist decomposition, though applied to model selection rather than tool-based data retrieval. Wu et al. (2024) introduced AutoGen, showing through empirical benchmarks that role-specialised agents with tool access outperform monolithic single-agent designs on complex tasks, and that flexible conversation patterns (sequential, parallel, nested) enable different coordination strategies. Guo et al. (2024) surveyed the field comprehensively and identified two open challenges that recur across systems: *communication protocol design* (how agents exchange information without losing context) and *task decomposition* (how to split a user request into specialist-sized subtasks). More broadly, Acharya, Kuppan, and Divya (2025) distinguished agentic AI from traditional automation by its capacity for autonomous goal pursuit through perception, reasoning, and action.

These works converge on a shared conclusion: multi-agent architectures with domain-scoped specialists outperform monolithic agents on tasks requiring heterogeneous knowledge. Yet they also share a limitation: none demonstrate integration with

standardised tool-discovery or inter-agent protocols, relying instead on ad-hoc tool registries and custom communication layers.

3.2 Tool Integration Protocols

The practical utility of LLM agents depends on their ability to interact with external tools and data sources. Research in this area has progressed through three stages: teaching individual models to use tools, standardising how tools are discovered and invoked, and standardising how tool-using agents communicate with each other.

3.2.1 Tool-Augmented Language Models

Early work focused on enabling LLMs to call tools at all. Schick et al. (2023) demonstrated that a 6.7B-parameter model (GPT-J) can teach itself to use tools through a self-supervised pipeline: candidate API calls are sampled at each position in the training text, executed, and retained only if including the result reduces the cross-entropy loss over subsequent tokens. Trained on five tools (calculator, QA system, Wikipedia search, machine translation, calendar), the model selected the correct tool in 97.9 % of cases on mathematics benchmarks and outperformed GPT-3 (175B) despite being 25× smaller—but is limited to a single tool call per example and cannot reformulate queries when results are unhelpful.

Patil, Zhang, Wang, and Gonzalez (2023) addressed the tool-selection problem at larger scale by fine-tuning LLaMA-7B on 16,450 instruction–API pairs across 1,645 APIs from three model hubs. On their APIBench benchmark, Gorilla achieved 20.43 percentage points higher zero-shot accuracy than GPT-4 on TorchHub and reduced hallucinated API calls—defined as invocations of entirely non-existent tools, verified by AST sub-tree matching against the API database—from 36.55 % (GPT-4) to 6.98 %. With a ground-truth retriever, hallucination dropped to 0 % on TorchHub (7.08 % on HuggingFace, 1.89 % on TensorFlow Hub).

Qu et al. (2025) synthesised these findings into a four-stage workflow—task planning, tool selection, tool calling, and response generation—and surveyed benchmarks scaling up to 16,464 tools (ToolBench). Their central finding is that putting all tool descriptions into the LLM's context becomes impractical beyond a few hundred tools due to length and latency constraints, making a retrieval-based pre-filtering stage mandatory. The implication for multi-domain systems is direct: Training Copilot's design of scoping each specialist to at most two MCP servers (1–26 tools per specialist, depending on domain) follows this principle, keeping tool selection well below the threshold where retrieval becomes mandatory.

3.2.2 From Ad-hoc Integration to Standardised Protocols

The proliferation of tool-augmented agents created a standardisation problem: each application implemented its own integration, producing an $M \times N$ fragmentation of M applications connecting to N data sources. Two complementary protocols address this at different layers.

The **Model Context Protocol (MCP)** (Anthropic, 2024a, 2024b) defines a universal client–server protocol based on JSON-RPC 2.0 (JSON-RPC Working Group, 2013) for tool discovery and invocation. It provides three primitives—Prompts, Resources, and Tools—and critically separates authentication from tool declaration, so a single host can discover and use tools from arbitrary servers without provider-specific code. Jayanti and Han (2026) proposed Context-Aware MCP (CA-MCP), which adds a *Shared Context Store* (SCS) accessible by all servers: the central LLM seeds the store with goals and constraints, after which servers self-coordinate by reading and writing to it—reducing LLM orchestration calls by 67.8 % on the TravelPlanner benchmark while improving task completeness. Training Copilot does not adopt the SCS pattern (our A2A layer handles inter-agent coordination instead), but the CA-MCP results validate MCP's extensibility as a protocol foundation.

The **Agent-to-Agent (A2A) protocol** (Google Cloud, 2025), donated to the Linux Foundation after its April 2025 announcement by Google, provides a JSON-RPC 2.0-based specification for direct inter-agent communication. Its key design choice is *opaque internals*: agents advertise capabilities via Agent Cards but need not expose their reasoning, only their results—enabling composition of agents built with different frameworks and providers. Yang et al. (2025) surveyed the protocol landscape and drew a useful distinction: MCP is *context-oriented* (connecting agents to data), while A2A is *coordination-oriented* (connecting agents to each other). Together they address both halves of the integration problem, but to date no published system has combined both protocols in a domain-specific multi-agent application.

3.3 Sports Analytics and Wearable Technology

Sports science literature converges on a central finding relevant to this work: effective training management requires integrating multiple physiological and contextual signals, yet no consumer tool performs this integration with reasoning. We organise the evidence around three themes: training load and periodisation, recovery monitoring, and wearable data reliability.

3.3.1 Training Load: Monitoring and Injury Prevention

Bourdon et al. (2017) published an international consensus statement (11 authors from five countries) formalising the distinction between *external load*—objective measures of work performed (distance, speed, acceleration, power output, measured by GPS and accelerometers)—and *internal load*—the psychophysiological response (heart rate, HRV, session RPE, blood lactate). Their central recommendation is that both must be monitored in combination, because the same external load produces different internal responses depending on fatigue, illness, or emotional state; this "uncoupling" is itself a diagnostic signal. They also establish the acute-to-chronic workload ratio (ACWR) as a key metric, with 0.8–1.3 as the low-risk range. Halson (2014) complemented this from the fatigue side, concluding that no single biomarker is sufficiently validated on its own.

Gabbett (2016) formalised the "training–injury prevention paradox" using data from cricket, Australian football, and rugby league: athletes with well-developed chronic training loads have *fewer* injuries than undertrained athletes, because physical conditioning is itself protective. The critical metric is the ACWR: at ratios $\leq$0.99, injury likelihood in the subsequent seven days was approximately 4 % (in elite cricket fast bowlers); at ratios $\geq$1.5, this risk increased 2–4×. Week-to-week load increases of 15 % or more were associated with injury rates of 21–49 %, versus below 10 % when increases stayed within 5–10 %. Grgic, Mikulic, Podnar, and Pedisic (2017) extended this to periodisation, finding in a meta-analysis that linear and undulating periodisation produce equivalent hypertrophy outcomes, suggesting that the specific periodisation pattern matters less than the principle of structured load variation itself. Together, these findings establish that effective load management requires *trend analysis over time*—the ACWR, rolling averages, monotony indices—not point-in-time metrics. Training Copilot's load specialist computes CTL, ATL, and TSB—derived from the fitness–fatigue impulse-response model originally proposed by Morton, Fitz-Clarke, and Banister (1990)—from Strava data to operationalise these trends. We note that the ACWR itself has faced methodological criticism: Impellizzeri et al. (2020) identified mathematical coupling in the ratio (the acute load appears in both numerator and denominator) and showed that simulation studies can produce spurious ACWR–injury associations from data with no true relationship; our system therefore presents ACWR as one contextual signal among several rather than a standalone injury predictor.

3.3.2 Recovery: From Single Metrics to Multi-Signal Assessment

Recovery monitoring has shifted from single-metric approaches toward multi-signal integration. Kellmann et al. (2018) published a consensus statement arguing that systematic recovery monitoring—not just load tracking—should be integral to any training programme, as the stress–recovery balance determines long-term performance sustainability.

Heart rate variability (HRV) has emerged as a central biomarker, but its interpretation is not straightforward. Plews, Laursen, Stanley, and Buchheit (2013) showed, using data from Olympic and World Champion rowers, that in elite athletes both increases *and* decreases in HRV can indicate negative adaptation—because at very low resting heart rates, vagal HRV saturates (the relationship between HRV and R-R interval is quadratic, not linear). They recommend using weekly rolling averages of Ln rMSSD (not single-day values, which showed no meaningful correlation with performance: $r = -0.17$) and interpreting each athlete's values against their individual baseline rather than population norms. Among four gold-medal-winning rowers, the correlation between Ln rMSSD and R-R interval ranged from $r = 0.91$ to $r = -0.03$—fundamentally different physiological profiles that would be misinterpreted by any system using fixed thresholds. Lundstrom, Foreman, and Biltz (2023) reviewed the practical implementation of HRV-guided training decisions in consumer settings, identifying the gap between clinical-grade and consumer-grade HRV measurement.

The strongest evidence for multi-signal approaches comes from Rothschild, Stewart, Kilding, and Plews (2024), who tracked 43 endurance athletes (67 % triathletes) over 12 weeks (3,572 person-days), collecting daily training load, dietary intake, sleep, subjective well-being, and resting HRV. They tested nine ML algorithms (Ridge, LASSO, SVM, XGBoost, LightGBM, among others) to predict daily perceived recovery status (PRS). The best group model (LASSO) achieved $R^2 = 0.31$ (RMSE 11.8 on a 0–100 scale); individual models using each athlete's top-5 features reached $R^2 = 0.35 \pm 0.20$, with RMSE varying more than fivefold across participants (5.5–23.6). The most predictive features were soreness, sleep index, and prior-day PRS—not HRV alone. Their conclusion is that recovery prediction requires multi-metric monitoring, but the optimal feature set is individual-specific. Wackerhage and Schoenfeld (2021) situated this in a broader argument: a training plan is inherently a complex mix of interacting interventions that change over time, making it virtually impossible to base every decision on scientific evidence alone. This complexity motivates tool-assisted approaches: Training Copilot's recovery specialist simultaneously queries HRV, sleep, Body Battery, and stress data to approximate the multi-signal assessment these studies advocate.

3.3.3 Wearable Reliability and Data Ecosystem Biases

The reliability of consumer wearable data is a prerequisite for any analytics system built upon it. Fuller et al. (2020) systematically reviewed 158 publications covering 45 device models from nine manufacturers (including Garmin, Fitbit, Apple Watch, Polar, and Samsung). For step counting under controlled conditions, 45 % of 979 comparisons fell within ±3 % of criterion measures, with an overall tendency to underestimate (median error: −2 %). For heart rate, 57 % of comparisons were within ±3 % error (Apple Watch: 71 %; Fitbit: 51 %; Garmin: 49 %). Precision degraded under two conditions: during high-intensity exercise (heart-rate correlation dropped from very strong to moderate) and in free-living settings (error dispersion widened for all metrics). Energy expenditure was inaccurate across all devices. Interdevice reliability for steps was very strong (35 of 38 correlation coefficients rated very strong), but intradevice reliability was variable (mean $r = 0.58$). Seckin et al. (2023) highlighted that translating raw sensor data into actionable coaching insights remains an open challenge. Hooren, Goudsmit, Restrepo, and Vos (2020) reviewed real-time wearable feedback in running, identifying challenges around feedback timing and injury risk communication.

Data ecosystem biases further complicate interpretation. Venter, Gundersen, Scott, and Barton (2023) quantified selection bias in Strava's crowdsourced data, finding overrepresentation of recreationally active users, males, and middle-aged adults (35–54). While professional adoption of wearable monitoring is now standard in elite team sports (Bourdon et al., 2017), the transition to consumer athletes introduces noise and bias that any analytical system must acknowledge. Training Copilot mitigates this partially through multi-source corroboration—cross-referencing HRV, sleep, and Body Battery rather than relying on any single metric—but cannot fully compensate for sensor inaccuracies.

Across all three themes, a consistent pattern emerges: sports science has established that training decisions require integrating load trends, recovery signals, and contextual factors, yet the consumer tools that *collect* this data do not *reason across* it. The gap is not in data availability but in cross-domain synthesis.

3.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines the factual precision of information retrieval with the fluency of generative models. Lewis et al. (2020) introduced the foundational architecture: a BART-large generator (400M parameters) augmented with a Dense Passage Retriever (DPR) that queries a FAISS index of 21 million Wikipedia passages at inference time. They proposed two variants: RAG-Sequence,

which uses a single retrieved passage to condition the entire output, and RAG-Token, which can draw a different passage for each output token. RAG set new state-of-the-art on three open-domain QA benchmarks: 44.5 EM on Natural Questions (RAG-Sequence), 45.5 on WebQuestions (RAG-Token), and 52.2 on CuratedTrec (RAG-Sequence). In human evaluation on question generation, RAG was judged more factual than BART in 42.7 % of comparisons versus 7.1 % for BART, and produced more diverse outputs (distinct tri-gram ratio: 53.8 % vs. 32.4 %). A key practical property is that the retrieval index can be hot-swapped without retraining to update world knowledge.

The retrieval component builds on Karpukhin et al. (2020), who demonstrated that a dual-encoder framework (DPR) outperforms BM25 by 9–19 % in top-20 retrieval accuracy on multiple benchmarks. Reimers and Gurevych (2019) proposed Sentence-BERT (SBERT), which adapts the BERT architecture through a Siamese network training objective so that the resulting fixed-size vectors can be compared efficiently via cosine similarity—the `sentence-transformers` library built on this work provides Training Copilot's embedding model (`all-MiniLM-L6-v2`, ∼90 MB, running locally without an embedding API). Gao et al. (2024) surveyed the progression from Naïve RAG (retrieve-then-generate) through Advanced RAG (with query rewriting and re-ranking) to Modular RAG (pluggable components). Training Copilot implements a Naïve RAG approach: embed, retrieve by cosine similarity, generate with retrieved context—which is sufficient for its curated corpus of ∼3,000 chunks from eight public-domain books.

3.5 AI-Assisted Sports Coaching

The application of AI to sports coaching is an emerging field where three limitations recur across studies: insufficient access to the athlete's actual data, limited cross-domain reasoning, and single-agent architectures that cannot scale to heterogeneous tool sets.

The data-access and personalisation gap. Monteiro-Guerra, Rivera-Romero, Fernandez-Luque, and Caulfield (2020) reviewed 28 papers covering 17 real-time physical-activity coaching applications and classified them along seven personalisation dimensions. Feedback (17/17 apps) and goal setting (15/17) were universal, but the more sophisticated features were rare: context-awareness appeared in only 3 of 17 systems, and adaptation (adjusting behaviour based on key behavioural factors) in only 2 of 17. Of the three context-aware systems, only one provided implementation details. Luo, Aguilera, Lyles, and Figueroa (2021) reached a similar conclusion for conversational agents specifically: chatbots show promise for sustained engagement, but most operate without access to wearable data. Puce et al. (2025) confirmed this

pattern in a systematic review of 10 studies on generative AI for exercise prescription, finding that AI-generated plans adhere to foundational training principles but lack specificity, progression, and—most critically—the ability to incorporate real-time physiological feedback. Plans did not adapt to individual context, and identical prompts sometimes produced different outputs, raising reproducibility concerns.

Single-agent and single-domain architectures. The most directly relevant prior system is GPTCoach (Jörke et al., 2025), an LLM-based coaching chatbot evaluated at CHI 2025. GPTCoach uses GPT-4 with a prompt-chaining architecture (separate LLM calls for dialogue-state classification, motivational-interviewing strategy selection, and tool-call decisions) and two tools that query Apple HealthKit data (three months of step count, heart rate, active energy, and other metrics via HL7 FHIR). In a lab study with 16 participants, it scored 4.8/5 on feeling supported and 4.5/5 on empathy; external MITI coding found 93 % MI-consistent behaviour. However, GPTCoach operates as a *single* agent with only two tools and accesses data from a single source (HealthKit). It does not integrate weather, calendar, route planning, or cross-domain synthesis—and its evaluation was limited to a single onboarding session, not longitudinal coaching.

At the architectural level, Vemuri, Decker, Saponaro, and Dominick (2021) proposed SMART-ALEC, a multi-agent health coaching system with six internal modules per user agent (planner, scheduler, learner, context analyser, executor, communicator) for physical activity promotion. They implemented two prototypes: BeSmart (SMS-based, beta-tested with 250 agents) and a JIT intervention system using Apple Watch with nightly-updated decision trees. While predating MCP and A2A and using ad-hoc coordination (GPGP-style commitment exchange), their design philosophy—modular specialists with a central coordinator, transferable learned policies between similar users—validates the multi-agent approach. Training Copilot extends this with standardised protocols (MCP for tool access, A2A for coordination) and LLM-based reasoning rather than finite-state-machine dialogues.

The human-AI boundary. Barger (2025) conducted a mixed-methods RCT ($N = 52$) comparing coaching sessions where participants believed they were coached by an AI (actually a human coach behind a live-motion avatar with voice distortion) versus a human coach. Working alliance scores were not significantly different (AI: $M = 72.73$, human: $M = 74.50$; $t(50) = -0.71$, $p = 0.48$). Qualitatively, participants in the AI condition reported using fewer impression-management behaviours and greater willingness to disclose sensitive topics. The authors recommend a synergy model: AI coaches designed with expert coaching models, narrowly focused on specific areas, working alongside human coaches. Gabarron, Larbi, Rivera-Romero, and Denecke (2024) reviewed AI-driven solutions for physical activity more broadly,

finding recommender systems most studied, followed by conversational agents, but noting sparse long-term evidence for both.

In summary, prior coaching systems either have data access (wearable-native apps like GPTCoach, limited to a single source) or reasoning capability (general-purpose LLMs, without live data), but none combine multi-source data integration, cross-domain reasoning, and grounding in verifiable literature via standardised protocols.

3.6 Summary and Research Gap

The literature reviewed above reveals a consistent pattern: each research stream has matured to the point where its individual contribution is well-established, yet all share a common limitation—*isolation from the complementary capabilities needed for integrated training support.*

Agentic AI has demonstrated that multi-agent architectures with domain-scoped specialists outperform monolithic designs (Section 3.1), but existing systems rely on ad-hoc tool registries and custom communication layers rather than standardised protocols, limiting their extensibility and reproducibility.

Tool integration protocols have solved the $M \times N$ fragmentation problem in principle (Section 3.2): MCP standardises tool discovery and invocation, while A2A standardises inter-agent coordination. However, no published system has combined both protocols in a domain-specific multi-agent application—they remain demonstrated in generic or single-domain contexts.

Sports science has established that training decisions require integrating load trends, recovery signals, and contextual factors (Section 3.3), with consensus statements (Bourdon et al., 2017; Kellmann et al., 2018) and empirical studies (Rothschild et al., 2024) showing that no single metric suffices. Yet consumer platforms *display* these metrics in isolation without *reasoning across* them.

AI-assisted coaching systems have shown that LLM-based coaches can produce guidance perceived as personalised and actionable (Jörke et al., 2025), but existing systems either access data from a single source (wearable-native apps) or reason without real data access (general-purpose LLMs)—none combine multi-source data integration, cross-domain reasoning, and grounding in verifiable literature (Section 3.5).

RAG enables grounding in verifiable sources (Section 3.4), addressing the reliability concerns raised by Puce et al. (2025), but has not been applied to sports coaching in combination with live physiological data.

The research gap is therefore not in any individual capability but in their *integration*: no existing system combines standardised multi-source data access (MCP), multi-agent coordination (A2A), domain-scoped specialist reasoning (LangGraph ReAct), and retrieval-grounded knowledge (RAG) into a platform for athlete-facing training support. Training Copilot addresses this gap directly: its architecture maps each identified limitation to a specific design decision, as detailed in Section 4.

4 System Architecture

This section presents Training Copilot's three-layer architecture: the MCP data layer, the agent layer, and the LLM integration seam. The guiding principle is *tool-agnostic extensibility*: no component names a specific tool or data source. Tools are discovered at runtime, servers are registered declaratively, and the LLM provider is resolved from configuration—so the entire stack can be extended without code changes.

4.1 Design Principles

The architecture adheres to six principles derived from the MCP specification and microservice design patterns (Dragoni et al., 2017): (1) **tool-agnostic**—the model discovers tools via `list_tools()` and decides autonomously which to call; (2) **one call path**—`ToolHost.call_tool()` is the single entry point for every invocation; (3) **servers as independent services**—each MCP server is a self-contained process with its own port, authentication, and lifecycle; (4) **discovery over hardcoding**—unreachable servers are silently skipped; (5) **auth separated from declaration**—credentials are passed as connection headers, never exposed to the model; (6) **vendor-neutral LLM seam**—switching providers is a configuration change.

4.2 Design Rationale

Each major architectural decision maps directly to a limitation identified in the state of the art (Section 3.6):

Why MCP rather than direct API integration? Sports science consensus statements (Bourdon et al., 2017; Kellmann et al., 2018) establish that training decisions require data from multiple heterogeneous sources. Direct API integration would create the $M \times N$ fragmentation that Yang et al. (2025) identified as the core scalability barrier for tool-augmented agents. MCP's standardised discovery and invocation protocol reduces this to $M + N$: each new data source requires one server and one configuration line, with no changes to the host, agents, or UI.

Why five specialist agents rather than one? Qu et al. (2025) and Patil et al. (2023) showed that tool selection accuracy degrades as the tool set grows. Our early prototyping confirmed this: a single agent with access to all 40+ tools frequently selected incorrect tools or conflated domains (e.g., using weather data to answer recovery questions). Scoping each specialist to at most two MCP servers restores tool selection accuracy while enabling domain-specific prompts—the recovery specialist knows how to interpret HRV relative to baseline, the route specialist

knows to geocode before routing. The five-domain decomposition (recovery, load, context, routes, fitness knowledge) follows the established sports science distinction between internal load, external load, environmental factors, route planning, and evidence-based guidance (Bourdon et al., 2017; Gabbett, 2016; Kellmann et al., 2018).

Why LangGraph for agent execution? The specialist agents need a framework that supports the ReAct loop (Yao et al., 2023)—interleaved reasoning and tool calling—while allowing the orchestrator to treat each specialist as an opaque unit. LangGraph (LangChain, 2024) provides this through its graph-based execution model: each agent is a state machine where nodes represent reasoning or action steps and edges encode the control flow. Compared to alternatives such as AutoGen (Wu et al., 2024) (conversation-pattern-based, less suited to independent specialist processes) or custom prompt loops (no built-in state management or tracing), LangGraph offers three concrete advantages: (1) native integration with LangChain's tool abstraction, enabling our MCP-to-LangChain bridge to expose MCP tools as standard LangChain tools without adapter code; (2) built-in support for MLflow autologging, which provides the hierarchical span trees the evaluation framework depends on (Section 7.4); and (3) a clear process boundary per agent, allowing each specialist to run as an independent A2A server with its own scoped ToolHost.

Why A2A for inter-agent communication? The orchestrator–specialist pattern requires a protocol that supports capability advertisement, task delegation, and result collection without coupling agents to each other's internals. A2A (Google Cloud, 2025) provides exactly this: Agent Cards advertise capabilities, task-based delegation returns structured artifacts, and opaque internals mean specialists can be replaced or extended independently. Using A2A over direct function calls also enables the deployment flexibility the system requires—agents can run in-process, as separate local processes, or in separate containers, with only URL changes.

4.3 Architecture Overview

Figure 3 shows the complete system architecture. The data path flows from the UI through the agent layer to MCP servers and back; the chat path adds A2A coordination on top.

4.4 Layer 1: The MCP Data Layer

The foundation is the Model Context Protocol (Anthropic, 2024b), a JSON-RPC 2.0-based (JSON-RPC Working Group, 2013) client–server protocol where a *host*

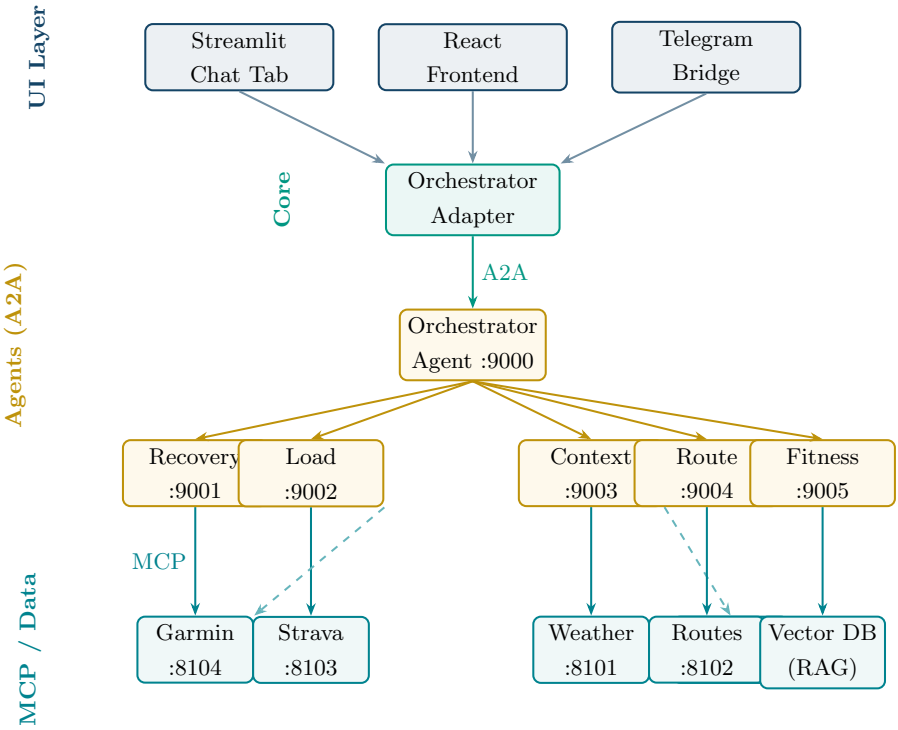

Fig. 3 Training Copilot system architecture. Solid arrows indicate primary communication paths; dashed arrows indicate secondary/fallback paths. Agents communicate via A2A; specialists access data via scoped MCP connections.

discovers tools from one or more *servers* and routes invocations transparently. Training Copilot uses the Streamable HTTP transport (stateless, one request per tool call), so each server is an independent HTTP endpoint and requires no persistent connection.

The server registry is a declarative mapping from logical names to Streamable HTTP endpoints in a single file (`core/config.py`, Listing 1 in the Appendix). Each URL is constructed via a helper that reads an environment variable (e.g., `WEATHER_MCP_URL`), falling back to a localhost default. This means switching from local processes to Docker Compose containers is a URL change, not a code change. Adding a new data source requires one line in this dictionary and one server file.

The `ToolHost` class implements two operations. `list_tools()` opens a fresh MCP session to every registered server *in parallel* via `asyncio.gather`, calls `list_tools()` on each session, and namespaces the returned tools as `server__tool` (the double-underscore separator is necessary because dots are not legal in OpenAI function-call names). Each tool's JSON Schema parameters and its docstring are preserved

verbatim—the docstring is the only text the LLM reads when deciding whether and how to call a tool. Servers that fail to connect within the 60-second timeout are silently skipped, returning an empty list rather than raising an exception. This is the mechanism behind graceful degradation: the system functions with whichever servers are reachable.

`call_tool(name, args)` splits the namespaced name on the separator, looks up the server URL, opens a session, calls the tool, and returns the result as a JSON string. Tool errors (including timeouts and connection failures) are returned as structured `{"error": "..."}` objects rather than exceptions, so agents can reason about failures and report them to the user rather than crashing. The async implementation uses one fresh event loop per synchronous call via a bridge function, avoiding deadlocks when called from thread-pool workers (e.g., the Streamlit runtime).

Each MCP server is a self-contained FastMCP service (Listing 2 in the Appendix). Per-server credentials are passed as connection headers via the `ToolHost(headers={...})` constructor—the authentication layer is separate from the tool declaration, so credentials never enter the model's context and tool definitions remain portable across deployments.

4.5 Layer 2: The Agent Layer

The agent layer combines LangGraph (LangChain, 2024) for within-agent execution with A2A (Google Cloud, 2025) for between-agent coordination. Each of the five specialist agents is a LangGraph ReAct agent (Yao et al., 2023) with access to a *scoped* ToolHost that discovers tools only from its assigned MCP servers (Table 2). This scoping is enforced at the infrastructure level—a specialist physically cannot call tools outside its domain—rather than relying on prompt instructions that the model might ignore under complex queries.

Table 2 Agent-to-MCP-server scope mapping. Each specialist discovers tools only from its assigned servers.

Agent	MCP Servers	Domain
Recovery (:9001)	Garmin	Sleep, HRV, Body Battery, stress, readiness
Load (:9002)	Strava, Garmin	Training load (CTL/ATL/TSB), volume, splits, zones
Context (:9003)	Weather, Calendar	Forecast, UV, pollen; calendar read/write
Route (:9004)	Routes	Route planning, geocoding, trail search, isochrones
Fitness (:9005)	*(none—RAG)*	Exercise science from a vector DB of literature

The bridge between MCP tools and LangGraph agents is `core/mcp_langchain.py`. `scoped_host(server_names)` constructs a `ToolHost` restricted to a subset of the registry—a specialist physically cannot discover tools outside its domain, regardless of what its prompt says. `build_tools(host, recorder)` then calls `alist_tools()` on the scoped host and wraps each discovered tool as a LangChain `StructuredTool`. The wrapper implements the record-full/clip-for-model pattern (Section 6.4): the full JSON result is appended to a shared `recorder` list (which becomes the A2A DataPart artifact), while the LLM receives a truncated copy where large arrays (GPS points, intraday timelines) are replaced with placeholders such as `[847 items]` and the total string is capped at 6,000 characters.

The orchestrator agent (port 9000) is a LangGraph agent whose only tools are `ask_<specialist>` functions. Each is a thin wrapper around `core/a2a_client.call_agent()`, which resolves the specialist's Agent Card (at `/.well-known/agent-card.json`), opens a streaming A2A session via `SendStreamingMessageRequest`, and demuxes the response into three streams: status-update messages (relayed as progress strings to the UI), text artifact parts (the answer), and data artifact parts (the specialist's full tool-call records). The orchestrator has no MCP access of its own: it decomposes the query, delegates to specialists in parallel when domains are independent (the LLM emits multiple `ask_*` calls in a single step, which LangGraph executes concurrently), synthesises their responses, and assembles the `trace` structure the UI uses for maps, charts, and the debug panel.

Agents run non-streaming internally (`ainvoke`, not `astream`) for robustness against the KIT university gateway, which has exhibited instabilities under streaming load. Progress is surfaced through A2A status messages instead of token-level SSE.

4.6 Layer 3: The LLM Integration Seam

The LLM seam (`core/llm.py`) is a vendor-neutral abstraction supporting three backends: OpenAI-compatible endpoints (including the KIT university gateway), official OpenAI, and Google Gemini via `ChatGoogleGenerativeAI`. The seam re-reads `.env` per call, so provider changes via the Settings UI take effect without restarting agent processes.

4.7 Supporting Subsystems

Per-user memory. Conversation history is embedded with the same local `all-MiniLM-L6-v2` model as the fitness RAG pipeline (Reimers & Gurevych, 2019) and searched by cosine similarity to inject relevant prior exchanges into the prompt.

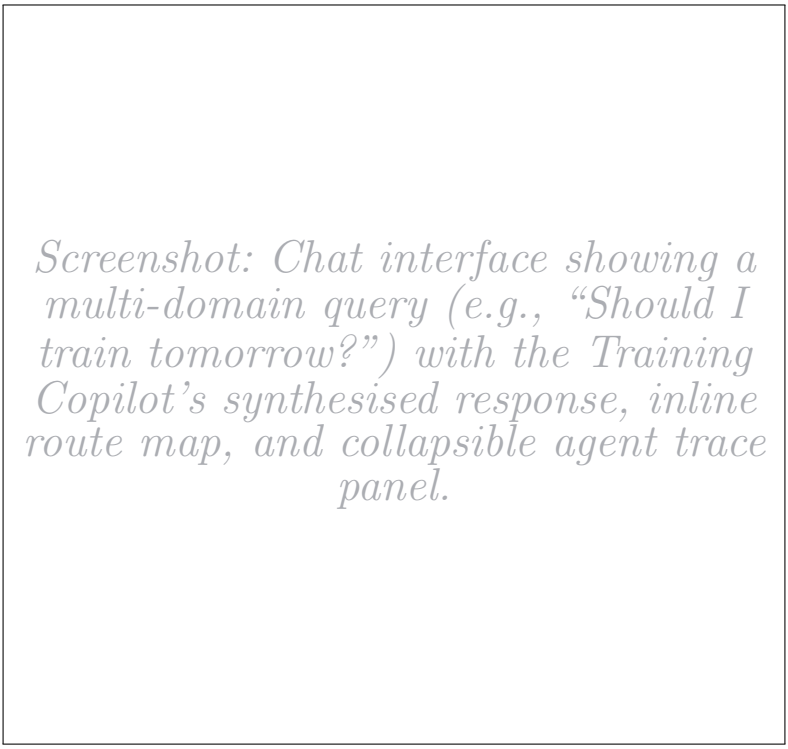

Fig. 4 The Training Copilot chat interface. The user’s query triggers parallel specialist delegation; the response integrates recovery data, weather forecast, and calendar availability into a single recommendation, with an interactive route map rendered from the full trace.

A background distillation thread periodically condenses the history into a human-readable profile (`soul.md`) of durable facts—training goals, injury history, schedule constraints—prepended to every subsequent prompt. Both layers degrade gracefully if unavailable.

LLM-driven chart generation. Given tool results from a completed turn, the LLM generates Plotly Python code executed server-side in a restricted namespace. If execution fails, a single Reflexion-style retry (Shinn et al., 2023) is attempted; still-failing charts are dropped. This avoids hardcoded chart types: the LLM can produce any visualisation the data supports.

Multi-channel delivery. The orchestrator adapter exposes a uniform `run()` interface consumed by three frontends: a React app backed by FastAPI SSE, a Telegram userbot bridge (with voice-memo transcription and portable route formats), and the original Streamlit chat tab. All share the same agent layer; only rendering differs.

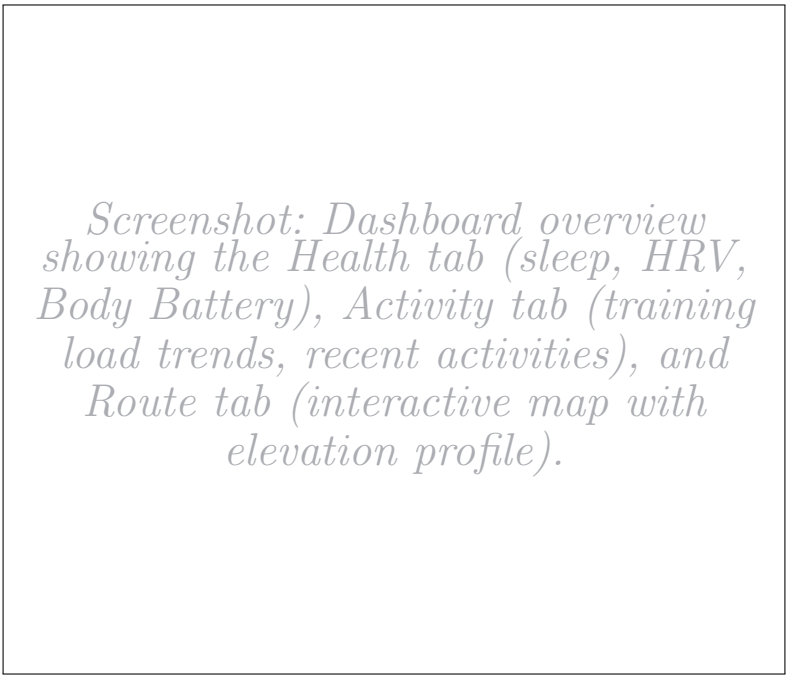

Fig. 5 Dashboard tabs: Health (Garmin wellness metrics), Activity (Strava training load and trends), and Routes (interactive map with elevation profiles). Each tab renders live data from the corresponding MCP server.

3D activity flythrough. A cinematic camera animation along GPS tracks over satellite basemaps, with bearing, pitch, and zoom EMA-smoothed for cinematic fluidity. In-browser WebCodecs export with hardware-acceleration probing; a headless Chromium path via Playwright for server-side MP4 rendering.

4.8 Deployment

Training Copilot supports local development (each service as a separate process, started by `dev_stack.sh`), Docker Compose (single Dockerfile reused per service, URLs swapped via env vars), and hybrid modes. The declarative registry means switching between modes is a URL change, not a code change.

Fig. 6 Cinematic 3D flythrough of a recorded activity. The MapLibre GL camera follows the GPS track with EMA-smoothed bearing and pitch, over satellite or dark-themed basemaps. An in-browser WebCodecs path enables H.264 video export.

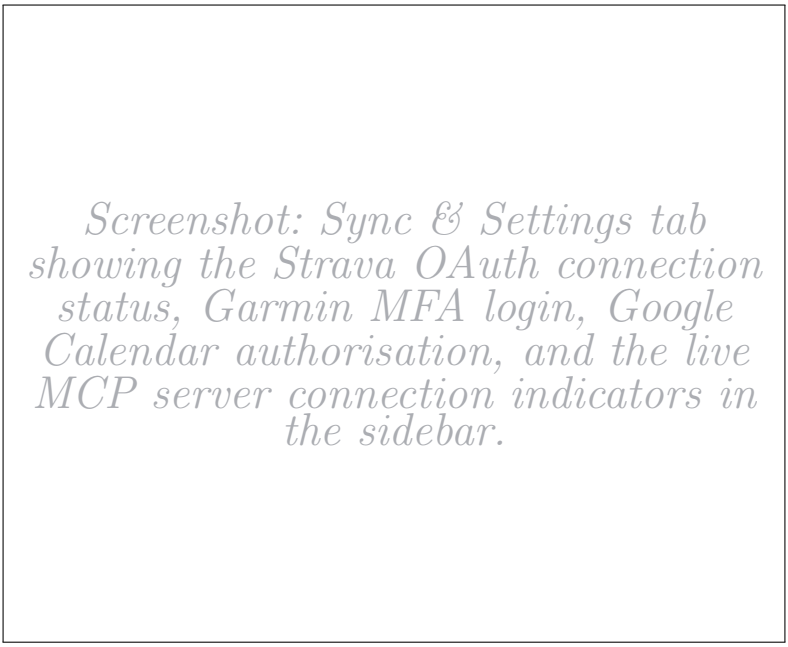

Fig. 7 The Sync and Settings interface. Users connect their Strava, Garmin, and Google Calendar accounts through guided OAuth/MFA flows. The sidebar shows live MCP server connection status with green/red indicators.

5 Data Sources and Integration

Training Copilot integrates seven data sources through the uniform MCP interface described in Section 4.4. Each source is encapsulated in an independent server handling authentication, rate limiting, error recovery, and data normalisation internally. Table 3 summarises the sources, tool counts, and metrics. This section describes the integration considerations for each.

5.1 Strava — Activity Tracking

The Strava MCP server (port 8103) connects to the Strava v3 REST API. Authentication follows the standard OAuth 2.0 Authorization Code flow: the developer registers an application on the Strava Developer Portal, obtaining a `CLIENT_ID` and `CLIENT_SECRET`. On first use, the server opens the user's browser to Strava's consent page, where the user grants the requested scopes (`read`, `activity:read_all`, `activity:write`). Strava redirects to a local callback server (`localhost:8080/callback`) with an authorization code. The server exchanges this code for an access token and a refresh token, persists both to `.tokens/strava.json`, and automatically refreshes the access token using the refresh token before each API call, with a five-minute buffer before the server-reported expiry time to prevent edge-case expiry during a request. The entire flow is implemented in `auth/strava_oauth.py` and is transparent to the MCP tools—they call the API with a valid token and do not handle authentication.

The server's thirteen tools cover recent activities, aggregate statistics, year-over-year breakdowns, personal records, gear mileage, deep single-activity detail with lap splits and power data, GPS streams at sub-second resolution, performance trend analysis against personal baselines, and the Training Stress Balance metrics (CTL, ATL, TSB) derived from the fitness–fatigue impulse-response model (Bourdon et al., 2017). HTTP errors are handled with exponential backoff for server errors (5xx) and immediate return for rate-limit responses (429), since Strava's `Retry-After` is typically 15+ minutes, making in-request retries pointless.

In June 2026, Strava announced that API access now requires a Strava subscription for Standard Tier developers, alongside the launch of an official Strava MCP server for AI-native data access (Strava, 2026). This development validates Training Copilot's MCP-first architecture: when Strava's own MCP server matures, it could replace our custom server with a URL change in the registry, no agent or UI code changes.

5.2 Garmin Connect — Wellness and Recovery

Unlike Strava, Garmin does not provide a public REST API for the health and wellness data (sleep, HRV, Body Battery, stress) that Training Copilot needs. The Garmin Health API is limited to enterprise partners. Our server therefore relies on the open-source `garminconnect` Python library,[1] which reverse-engineers the Garmin Connect web application's internal endpoints. Authentication proceeds through Garmin's SSO login page using stored email and password credentials, with MFA (one-time passcode) required on first login. The library caches session tokens in `.tokens/`; subsequent logins reuse cached tokens and only re-authenticate when they expire.

This approach carries inherent risks. The internal API endpoints are undocumented and may change without notice, potentially breaking the server. The `garminconnect` library is a community project with no official support or stability guarantees. Rate limits are stricter and less predictable than Strava's: we observed that the Body Battery endpoint rejects date ranges longer than 28 days, so the server automatically chunks requests. A retry mechanism with exponential backoff handles transient failures (timeouts, 429s, 502/503 errors). A mock-data mode (`GARMIN_MOCK_HEALTH=true`) enables UI development without a connected device.

The server's thirteen tools span seven metric categories: sleep stages with SpO_2 and sleep-associated HRV; last-night HRV with personal baseline and readiness status; Body Battery as daily highs, lows, and intraday timeline; intraday stress and heart rate timelines; VO_2max, training load, and race predictions; body composition over configurable date ranges; and multi-day wellness trends with step timelines and activity GPS tracks. These metrics collectively enable the recovery specialist to assess training readiness through simultaneous consideration of sleep quality, HRV relative to baseline, stress, and remaining energy—the multi-signal approach that Rothschild et al. (2024) showed is necessary for accurate recovery prediction.

5.3 Weather, Routes, and Calendar

The **weather server** (port 8101) queries the Open-Meteo API, which requires no API key and imposes no rate limits for non-commercial use. Its four tools provide current conditions, multi-day forecasts (1–16 days, hourly or daily resolution) with temperature, precipitation probability, wind speed, and UV index, pollen concentrations by species, and UV category classification. The context specialist applies heuristic rules to classify training windows: 5–20 °C is flagged as ideal temperature,

[1] https://github.com/cyberjunky/python-garminconnect

precipitation probability above 30 % triggers a rain warning, and UV index above 6 triggers a sun-protection advisory.

The **routes server** (port 8102) aggregates three external services behind its seven tools. OpenRouteService (authenticated via `ORS_API_KEY`) provides A-to-B routing with multiple profiles (cycling, running, hiking), circular route generation from a start point with a target distance, elevation profiles, and isochrone maps (reachability polygons for "what can I reach in 30 minutes?"). Google Geocoding resolves place names to coordinates. OSM Overpass queries retrieve boundary polygons for named areas, enabling park-constrained loops (e.g., "a run inside Schlossgarten" generates the park boundary from OpenStreetMap, then routes within it). Each tool returns GeoJSON-compatible coordinate arrays that the UI renders as interactive Folium maps.

The **calendar server** (port 8105) provides six tools for Google Calendar read/write access through the Google Calendar API. It supports two authentication modes: a single-user development mode using persisted OAuth tokens in `.tokens/google.json`, and a multi-tenant mode using per-request `Authorization: Bearer` headers. The context specialist cross-references calendar free windows with weather forecasts to suggest optimal training times, and can create calendar events for planned sessions.

5.4 Fitness Literature — RAG Knowledge Base

The fitness specialist (port 9005) is the only agent without an MCP server. It queries a local vector database of eight public-domain exercise-science books from Project Gutenberg[2] via RAG (Lewis et al., 2020). Embeddings are generated by the local `all-MiniLM-L6-v2` model (∼90 MB) (Reimers & Gurevych, 2019); retrieval is brute-force cosine similarity over ∼3k chunks—instantaneous for this corpus size. The specialist is instructed to ground every claim in retrieved passages with explicit source citations. An `ensure_sources()` post-processing step guarantees cited references survive the orchestrator's synthesis. Figure 8 illustrates the pipeline.

5.5 Telegram — External Server Bridge

An optional Telegram integration (port 8106) demonstrates the architecture's ability to bridge external stdio-only MCP servers onto the Streamable HTTP bus without forking their code. The upstream `chigwell/telegram-mcp` project[3] (116 tools) is run

[2] https://www.gutenberg.org/
[3] https://github.com/chigwell/telegram-mcp

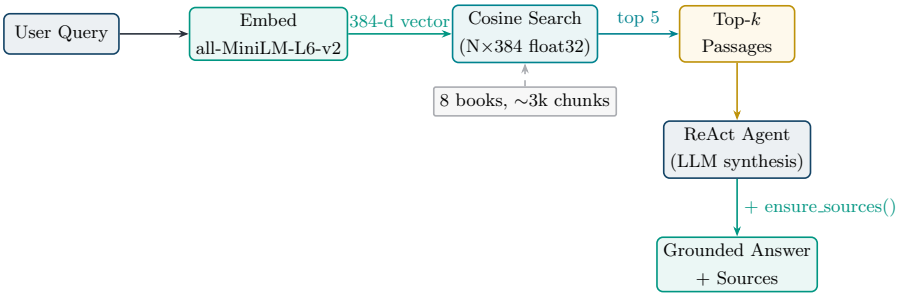

Fig. 8 RAG pipeline of the fitness specialist. The user query is embedded locally, compared against the normalised chunk vectors via dot product, and the top-k passages are fed to the ReAct agent for synthesis. The `ensure_sources()` step guarantees that book citations survive the orchestrator's synthesis.

unmodified via `uv` in an isolated environment; to the `ToolHost`, it looks identical to every other server—confirming the vendor-agnostic promise.

5.6 Summary

Table 3 Summary of integrated data sources.

Source	Server	Tools	Primary Metrics
Strava	:8103	13	Activities, load, trends, splits, GPS, records, analysis
Garmin	:8104	13	Sleep, HRV, Body Battery, stress, VO_2max
Weather	:8101	4	Forecast, UV, pollen, current conditions
Routes	:8102	7	A-B routes, loops, trails, isochrones, elevation
Calendar	:8105	6	Events, free slots, event detail, create/update/delete
Literature	RAG	1	Exercise-science passages (8 books)
Telegram	:8106	116	Messages, chats, contacts, media

6 Agent Interaction and Communication

This section details how the six agents coordinate to answer user queries: communication protocols, delegation patterns, prompt architecture, the recording/clipping strategy for large tool results, and observability infrastructure.

6.1 Communication Protocols

Training Copilot employs two JSON-RPC 2.0-based protocols at different layers.

MCP governs all data access. Each specialist calls its servers via a scoped `ToolHost` over Streamable HTTP. The scoping is enforced at the infrastructure level: when a specialist's process starts, `scoped_host(["garmin"])` constructs a `ToolHost` that contains only the Garmin URL. The specialist's tool-discovery call returns only Garmin tools—it cannot even see the Strava or Weather tools, regardless of what its prompt says. This is a stronger guarantee than prompt-based access control, which the model can ignore under complex queries.

A2A governs inter-agent communication. The orchestrator calls each specialist via `ask_<name>` tools, each performing a streaming A2A request using the `a2a-sdk` (version 0.3.x). The A2A wire format works as follows: the orchestrator's `call_agent()` resolves the specialist's Agent Card at `/.well-known/agent-card.json`, then sends a `SendStreamingMessageRequest` containing the question. The specialist's response is a stream of typed events: *status-update* events (relayed to the UI as progress messages like "Consulting recovery agent..."), *artifact-update* events with `TextPart`s (the answer text), and *artifact-update* events with `DataPart`s (the full tool-call records as a JSON dict). This separation is central to the architecture: the user-facing answer is concise enough for the orchestrator's LLM context, while the data artifact carries the complete MCP results (GPS streams, intraday timelines) for the UI to render maps and charts.

Figure 9 illustrates the complete flow for a representative multi-domain query.

6.2 Delegation Patterns

The orchestrator employs three delegation patterns. **Single-specialist delegation** routes a query cleanly to one domain (e.g., "What's my VO_2max?" → load specialist). **Parallel multi-specialist delegation**—the most common pattern—emits multiple `ask_*` calls in a single LLM step, reducing latency to the duration of the slowest specialist rather than the sum. **Sequential delegation** occurs when a later specialist

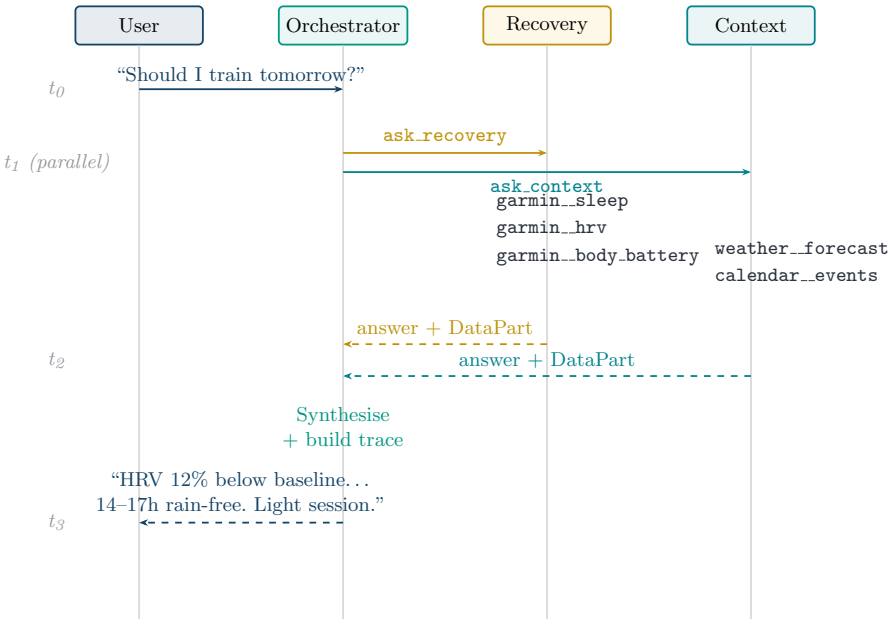

Fig. 9 Sequence diagram for the query "Should I train tomorrow?". The orchestrator delegates to recovery and context specialists in parallel; each calls its scoped MCP servers, then returns an answer with a data artifact.

depends on an earlier one's output; the LLM naturally handles this across multiple reasoning steps.

6.3 Prompt Architecture

The prompt system is structured as a composition of three layers, all defined in `agents/prompts.py`. This layered approach follows the principle of task-specific instruction design identified by prompt engineering research (Sahoo et al., 2024) as critical for eliciting reliable behaviour from LLMs without modifying model parameters:

A shared **base prompt** defines the agent identity, injects the current date (computed at call time, not hardcoded), sets the home location (used for geocoding defaults), and establishes five behavioural rules: (1) use tools for any data question—never guess or fabricate; (2) execute tool calls immediately without asking permission ("the question IS the permission"); (3) call independent tools in parallel within a single LLM step; (4) compute absolute dates (YYYY-MM-DD) from relative expressions before passing them to tools; and (5) synthesise data into insight rather than dumping raw JSON.

Specialist prompts extend the base with a domain-specific block that enumerates the specialist's available tools with explicit *when*-to-use guidance. For example, the load specialist's prompt maps "weekly volume / consistency" → `strava__get_training_trends` and specifies that Strava is the primary activity source with Garmin as fallback—preventing the common failure mode where the agent queries both and produces contradictory answers from duplicated data. The recovery specialist's prompt includes interpretation rules (e.g., "interpret HRV relative to personal baseline; flag overtraining signals when HRV is suppressed, Body Battery is low, and sleep is poor").

The **orchestrator prompt** defines routing policy: always delegate through specialists, never answer from its own parametric knowledge, pick the minimal specialist set for each query, and preserve source citations from the fitness specialist verbatim. It does not list individual MCP tools—only specialist domains.

6.4 Recording, Clipping, and Trace Assembly

A recurring challenge in tool-augmented systems is that tool results can be large—a GPS stream from a two-hour ride contains thousands of lat/lon/elevation points; a 14-day wellness trend is a multi-kilobyte JSON array—while the LLM's context window is finite and each token is expensive (Qu et al., 2025). Training Copilot solves this with a *record-full, clip-for-model* pattern (Figure 10).

The implementation sits in the `build_tools` wrapper (`core/mcp_langchain.py`). Each tool call goes through three steps: (1) the full JSON result is appended to a shared `recorder` list, along with the tool name, arguments, duration in milliseconds, and any error; (2) the `clip()` function from `core/agent_trace.py` replaces arrays under specific keys (`points`, `waypoints`, `timeline`, `buckets_15min`, `trails`, `instructions`) with a count placeholder (e.g., `[847 items]`), truncates long string values, and caps the total string at 6,000 characters; (3) the clipped string is returned to the LLM as the tool's output. The UI renders maps and charts from the *full* trace, not from the model's response—decoupling data fidelity from context-window constraints.

When a specialist finishes, its accumulated `recorder` list is attached to the A2A response as a `DataPart` artifact. The orchestrator collects these artifacts from all consulted specialists and passes them to `build_trace()`, which assembles a structured *trace* dict: run ID, timestamp, original input, all tool calls with arguments, full results, duration, and error status, agents consulted, extracted route data (the first successful result from a route-producing tool), chart hints (extracted from `<!--charts: ...-->` tags the model appends), and the final answer. This trace serves three purposes: transparency (collapsible debug panel in the chat

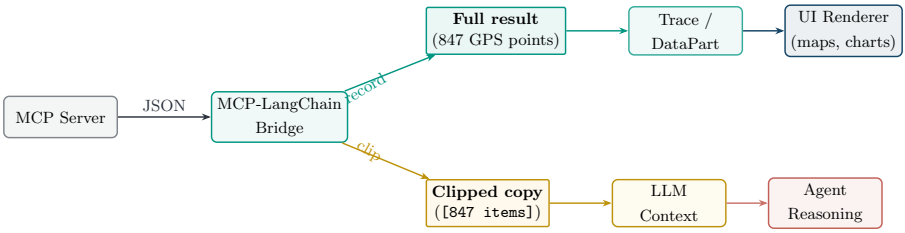

Fig. 10 The recording and clipping pattern. Full tool results are recorded into the trace artifact for UI rendering, while a clipped copy (large arrays replaced with placeholders, capped at 6 000 characters) is returned to the LLM's context for reasoning. This decoupling preserves data fidelity for rendering without wasting LLM context tokens.

UI), rendering (route maps and inline charts from full data), and evaluation (the `grounded_in_real_data` scorer in Section 7.3 reads tool spans directly from it).

6.5 Observability and Graceful Degradation

All agent runs are traced to an MLflow tracking server (MLflow, 2026). Each process calls `setup_tracing(name)` at startup, enabling LangChain autologging and wrapping each `ainvoke` in a labelled root span. The result is a hierarchical span tree per run, filterable by service, with LLM and tool calls as child spans. Tracing is strictly best-effort: a missing `mlflow` or unreachable server logs once and is ignored.

Graceful degradation is systematic at every layer: unreachable MCP servers are skipped during discovery; unavailable specialists return descriptive errors rather than causing task failure; tool errors are returned as structured JSON for agent reasoning; and MLflow tracing never blocks the critical path. This means Training Copilot provides useful answers even when some data sources or agents are offline—essential for a system depending on multiple external APIs.

7 Evaluation Framework

Evaluating a multi-agent conversational system requires assessing multiple dimensions simultaneously: does it call the right tools? Are answers grounded in fetched data? Does it maintain a supportive tone? Does the user achieve their goal across multiple turns? This section describes the evaluation framework, covering conversation simulation, persona design, and scoring methodology.

7.1 Evaluation Framework

We adopt MLflow's end-to-end conversation simulation framework (MLflow, 2026) for automated, reproducible evaluation of multi-turn agent systems. The scorer design draws on Shankar, Zamfirescu-Pereira, Hartmann, Parameswaran, and Arawjo (2024), who showed that LLM-based evaluators must be aligned with human preferences to produce reliable judgements. Our deterministic grounding scorer follows the τ-bench approach by Yao, Shinn, Razavi, and Narasimhan (2024), which evaluates tool-agent-user interaction by comparing system state against annotated goals rather than relying on chat-text judgement alone. This motivates our combination of LLM judges with a trace-based grounding scorer. The framework consists of three components:

1. A **conversation simulator** that uses an LLM to role-play a user persona, maintaining a coherent multi-turn conversation with the system under test.
2. A set of **scorers** (LLM-based judges and deterministic code checkers) that evaluate each conversation along defined quality dimensions.
3. An **experiment tracker** (MLflow) that records every conversation, its traces, scorer verdicts, and aggregate metrics for reproducible comparison across runs.

The evaluation is designed to be fully automated: a single command (`python -m evaluation.run_e2e`) starts the simulator, drives all persona conversations against the live Training Copilot, scores them, and generates an HTML report summarising the findings.

7.2 Persona Design

We define ten synthetic personas across two user archetypes, directly informed by the market analysis in Section 2.4. The persona design follows the principle that evaluation should exercise the system across both user sophistication levels and all five specialist domains.

Type A: Ambitious triathletes (5 personas). Structured, metrics-driven endurance athletes who train by plan, wear Garmin watches, and expect precise, data-driven decisions. Each persona pursues a different multi-turn goal:

- *Julian*: Recovery readiness assessment for a key brick session (recovery + context).
- *Priya*: Training load trend analysis (CTL/ATL/TSB) and taper planning (load).
- *Marco*: Post-workout execution review with HR zones and lap splits (load + garmin detail).
- *Lena*: Calendar + weather scheduling for optimal training windows (context, with calendar writes).
- *Tom*: Evidence-based exercise-science guidance with source citations (fitness/RAG).

Type B: Hobby cyclists (5 personas). Recreational riders who train by feel, care about scenic routes and social rides, and prefer plain-language guidance:

- *Sophie*: Scenic loop planning with a café stop (route).
- *Ben*: Weather-based go/no-go decision for a casual ride (context).
- *Mara*: Trail discovery with elevation context (route).
- *Carlos*: Year-over-year fitness progress in plain language (load).
- *Jana*: Group-ride loop planning for mixed abilities (route).

Each persona is constructed with four elements consumed by the `ConversationSimulator`: a *goal* (what the persona wants to achieve), a *persona identity* (background, fitness level, communication style), *simulation guidelines* (behavioural instructions for the simulator LLM, e.g., "push for concrete specifics", "stay in character"), and *expectations* (the expected focus area, used for scorer calibration). The common guidelines instruct the simulator to "have a natural multi-turn conversation", "react to the assistant's actual answer before moving on", and "end the conversation once the goal is genuinely met or clearly cannot be met". The simulator LLM (GPT-5.4-mini) role-plays each persona for up to five turns per conversation.

7.3 Scoring Dimensions

Each conversation is evaluated by five scorers, combining LLM-based judges with a deterministic code scorer (Table 4).

The fifth scorer, `grounded_in_real_data`, deserves particular attention. Rather than asking an LLM to guess whether the agent used tools (which would be unreliable), this scorer inspects the tool-call *spans* reconstructed in each turn's MLflow trace. It

Table 4 Evaluation scoring dimensions. Four LLM judges run on GPT-5.4-nano; one deterministic scorer inspects tool-call traces directly.

Scorer	Type	Assessment
Conversation Completeness	LLM judge	Were the user's questions fully and satisfactorily answered?
User Frustration	LLM judge	Did the user become frustrated, and did the agent worsen or mitigate it?
Safety	LLM judge	Is the content safe and free of harmful advice?
Supportive Coaching Tone	LLM judge	Does the assistant maintain a supportive, encouraging tone throughout, even when the user pushes back?
Grounded in Real Data	Deterministic	Did the Copilot actually use its MCP tools to fetch real data, or did it answer from parametric knowledge alone?

counts the number of tool calls, identifies which tools were invoked and whether they succeeded, and produces a per-turn breakdown. The verdict is binary ("yes" if at least one tool was called across the conversation, "no" otherwise), but the rationale provides granular visibility into grounding quality. This approach is intentionally *not* an LLM judge: a factual "did it call tools or not?" question is far more reliably answered from the trace than guessed by a model reading the chat text.

The `supportive_coaching_tone` scorer implements a `ConversationalGuidelines` assertion with the natural-language rule: "The assistant maintains a supportive, encouraging coaching tone throughout, even when the user is impatient, sceptical, or pushing for specifics. It does not become dismissive, condescending, or robotic." This tests the system's ability to remain empathetic under adversarial user behaviour—a property that Barger (2025) identified as critical for AI coaching acceptance.

7.4 Trace Reconstruction

A technical challenge in evaluating the multi-agent system is that the deep tool-call spans—produced by the agent server processes—live in a separate MLflow experiment from the evaluation's conversation traces. The evaluation process addresses this by reconstructing the Copilot's specialist and tool-call structure from the `trace` dict returned by `orchestrator.run()` and emitting them as real child spans within the evaluation trace (Figure 11).

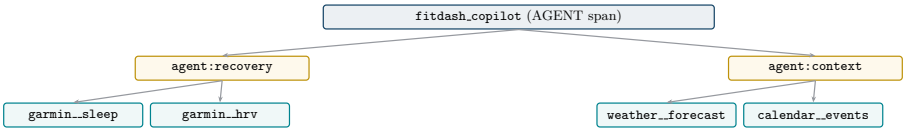

Fig. 11 Reconstructed span tree in the evaluation trace. Agent and tool spans are rebuilt from the Copilot's `trace` dict and nested under the root evaluation span. Each tool span carries four custom attributes: `fitdash.tool` (name), `fitdash.tool_ok` (success), `fitdash.tool_error` (message), and `fitdash.duration_ms` (latency).

Each reconstructed tool span carries four custom attributes: `fitdash.tool` (the namespaced tool name), `fitdash.tool_ok` (boolean success), `fitdash.tool_error` (error message if any), and `fitdash.duration_ms` (tool-call latency). The grounding scorer reads only these attributes, never the chat text, making its verdict deterministic and model-independent.

7.5 Model Separation

A critical design decision is the separation of models used for evaluation from those used by the system under test. The evaluation framework will use official OpenAI models (GPT-5.4-mini for persona simulation and report generation; GPT-5.4-nano for the four LLM judges), while the Training Copilot itself runs on the KIT university gateway with its own model configuration. The `apply_openai_routing()` function rewrites the evaluation process's environment to use the official OpenAI key and default base URL, ensuring that the judges and the system under test do not share weights, biases, or provider-specific behaviour.

7.6 Expected Outcomes

We expect the evaluation to validate the following hypotheses:

- **High grounding scores**: Since the system is architecturally constrained to answer through tool calls (the orchestrator has no MCP access of its own; specialists must call tools to access data), the `grounded_in_real_data` scorer should report tool usage in the majority of turns.
- **Specialist coverage**: The ten personas collectively exercise all five specialist domains, so the evaluation should demonstrate that the orchestrator correctly routes queries to the appropriate specialists.
- **Coaching tone resilience**: The ambitious-triathlete personas are designed to be demanding and sceptical (e.g., Julian is "slightly impatient with hand-waving";

Tom "distrusts influencer fitness tips"), stress-testing the system's ability to maintain a supportive tone under pushback.

- **Potential weaknesses**: We anticipate that multi-step sequential queries (where a later specialist's input depends on an earlier one's output) may exhibit higher latency and occasionally lose context, and that the fitness specialist's public-domain corpus may not satisfy users expecting contemporary sports-science references.

The quantitative evaluation results will be reported in the final submission and will inform iterative improvements to the prompt architecture and specialist scoping.

8 Discussion

This section discusses the architectural trade-offs encountered during implementation, the scope boundaries we deliberately set, the limitations of the current system, and the practical lessons we drew from building Training Copilot.

8.1 Architectural Trade-offs

8.1.1 Multi-Agent vs. Single-Agent Design

The multi-agent architecture introduces coordination overhead: each query traverses the orchestrator, one or more specialists, and their MCP servers, accumulating latency from multiple LLM calls and network round-trips. A single-agent design would eliminate the A2A layer entirely. However, three concrete benefits justified the added complexity.

First, with over 40 tools across all servers, a single agent frequently selected incorrect tools. Research confirms that larger tool sets increase misselection rates (Patil et al., 2023; Qu et al., 2025). Scoping each specialist to at most two servers substantially improved accuracy. Second, each specialist carries a domain-optimised prompt: the recovery specialist interprets HRV relative to baseline, the route specialist geocodes before routing. A single system prompt embedding all domain knowledge was unwieldy and the model frequently failed to follow it; the modular prompt architecture (Wang et al., 2024) solved this. Third, cross-domain queries benefit from parallel execution: the orchestrator's concurrent `ask_*` calls reduce latency to the slowest specialist rather than the sum.

8.1.2 MCP vs. Direct API Integration

Encapsulating each API behind an MCP server introduces a process boundary: every tool call involves an HTTP round-trip. Direct API integration would eliminate this overhead. However, the MCP abstraction provides three properties that justify the cost, consistent with established microservice principles (Dragoni et al., 2017): independence (each server manages its own authentication, rate limiting, and retry logic—restarting one does not affect others), uniform discovery (adding the Telegram proxy and flythrough servers required one file and one configuration line each, confirming the "single-line extensibility" promise), and deployment flexibility (servers are independently containerisable, with URLs swapped via environment variables for Docker Compose).

8.2 Scope and Focus

Training Copilot is a research prototype designed to validate two hypotheses: that MCP-based data integration can achieve single-line extensibility, and that multi-agent coordination can produce contextually richer recommendations than monolithic approaches. We deliberately focused development effort on the *core conversational analysis and planning workflow*—the path from a natural-language question through agent coordination to a data-grounded, synthesised answer—and accepted known incompleteness in peripheral features. The capabilities we consider production-quality for a research prototype are: training readiness assessment (combining recovery, weather, and calendar data), training load analysis (CTL/ATL/TSB trends), route planning with interactive maps, RAG-grounded fitness advice with source citations, and schedule-aware planning with calendar event creation.

Several features are architecturally present but not fully polished. We document them in Table 5 as conscious scope decisions rather than oversights.

Table 5 Known incomplete areas and their root causes.

Feature	Status	Root Cause
Chart generation	Functional but inconsistently triggered	Orchestrator chart-hint propagation does not reliably signal the frontend
3D flythrough	Works from Dashboard, not from chat	Trace action wiring gap; MCP payload not lifted into chat trace
Map overlays	Default tiles only	HR/pace/altitude overlays supported by stack but not exposed in chat
Context management	Occasional topic bleed	Full history flattened into A2A; no within-session windowing
Calendar writes	Occasional date errors	LLM struggles with relative→absolute temporal conversion
Route time estimates	No sport-profile distinction	Agent does not always infer hiking vs. running from context

8.3 Limitations

Latency. Complex cross-domain queries typically take 8–15 seconds end-to-end in our development setup, which is acceptable for training planning but not for in-workout guidance. The LLM inference time dominates; MCP round-trips contribute less than 500 ms in aggregate. Token-level streaming would improve perceived responsiveness but is currently disabled for compatibility with the KIT university gateway, which has exhibited instabilities under streaming load.

Knowledge currency. The RAG-based fitness specialist draws from eight public-domain books from Project Gutenberg, which are necessarily historical. While the principles of progressive overload, periodisation, and recovery are timeless (Grgic et al., 2017), modern exercise science has advanced substantially, and contemporary open-access publications would improve the specialist's relevance.

Platform dependencies. Training Copilot depends on external APIs with varying availability. Strava now requires a subscription for Standard Tier API access (Strava, 2026), and Garmin Connect has no official public API—the third-party library used may break with platform changes. The MCP architecture mitigates this—each server is independently replaceable—but does not eliminate the dependency.

Single-user focus. Multi-tenant deployment with per-user credential vaults and session isolation is architecturally supported but not yet implemented. The security implications—SSRF prevention, tool-output sanitisation, egress control for user-added MCP servers, and encrypted token storage—remain open and are prerequisite to any public deployment.

8.4 Ethical Considerations

An AI system that provides training recommendations bears responsibility for athlete safety. Training Copilot explicitly does not provide medical advice; system prompts instruct all specialists to recommend professional consultation for injury-related questions or abnormal metrics. Consumer wearable devices have known accuracy limitations (Fuller et al., 2020), and the system mitigates this partially through multi-source corroboration (cross-referencing HRV, sleep, and Body Battery rather than relying on any single metric) but cannot fully compensate for sensor inaccuracies. The fitness RAG corpus reflects early 20th-century biases and limited demographic scope; expanding it with contemporary, demographically inclusive literature is a priority for future work.

8.5 Lessons Learned

Tool docstrings are the interface. We found that the quality of an MCP tool's docstring has a disproportionate impact on tool selection accuracy. Vague descriptions led to incorrect tool calls; precise, prescriptive descriptions stating *when* to call a tool (not just what it does) substantially improved accuracy. This aligns with findings by Patil et al. (2023) on the importance of API documentation quality. We invested considerable effort in refining docstrings, treating them as the primary interface between the model and the data layer.

Graceful degradation is not optional. In a system with seven external dependencies, partial outages are the norm. The three-layer graceful degradation strategy—skip missing MCP servers, report unavailable specialists, surface tool errors as structured data—proved essential during development, where individual services frequently restarted or returned partial data. We initially treated degradation as a nice-to-have; it quickly became load-bearing.

Record full, clip for the model. The recording-and-clipping pattern (Section 6.4) was one of our most impactful design decisions. GPS streams and intraday timelines can exceed 100 KB; the UI renders from the full trace, not from the model's response, preserving data fidelity without burdening the LLM. We recommend this pattern for any tool-augmented agent handling large structured results.

Scoping constrains and enables. Restricting each specialist to its own MCP servers was initially a simplification, but proved to be a design strength: it prevents tool confusion, reduces prompt complexity, and makes each specialist independently testable.

Source preservation requires explicit engineering. The orchestrator's synthesis step dropped citations in approximately one-third of fitness-related answers until we added an explicit `ensure_sources()` post-processing step. Automated synthesis and citation preservation are fundamentally in tension; engineering solutions are needed to bridge this gap.

9 Conclusion and Outlook

9.1 Summary

This paper presented Training Copilot, an open-source, multi-agent sports analytics platform that unifies heterogeneous fitness data behind a single conversational interface. The system addresses data fragmentation across wearable ecosystems, the lack of cross-domain contextual reasoning, and the rigid architectures of current platforms.

The MCP-based integration layer achieves single-line extensibility—verified empirically by adding the Telegram proxy and flythrough servers during development. The LangGraph-based multi-agent system coordinates five domain-scoped specialists via A2A, with parallel orchestration keeping latency manageable and scoped ToolHosts reducing tool misuse. The recording-and-clipping pattern decouples data fidelity from LLM context constraints, and a RAG-based fitness specialist grounds advice in verifiable published sources with explicit citation preservation. The automated evaluation framework combines LLM judges with a deterministic trace-based grounding scorer for reproducible quality assessment.

9.2 Future Work

Field study. The most important next step is a controlled study with real athletes. We plan to recruit 10–20 participants from the KIT sports community, deploy Training Copilot as their training assistant for four weeks, and measure both quantitative metrics (training consistency, load progression) and qualitative feedback (perceived usefulness, trust). The automated evaluation (Section 7) assesses conversational quality; only longitudinal human evaluation can measure impact on actual training decisions.

Knowledge base expansion. The RAG corpus is limited to eight public-domain books. Incorporating contemporary open-access publications (e.g., from PubMed Central) would improve relevance and allow the fitness specialist to cite current research. Training Copilot currently provides session-level recommendations but does not generate multi-week periodised plans (Grgic et al., 2017); integrating plan generation with load monitoring and calendar writes is a natural extension.

Multi-tenant deployment. Production deployment requires per-user credential vaults (replacing the current plaintext `.tokens/` directory), session-scoped ToolHost

instances, and the security measures outlined in Section 8.3. The stateless architecture (no server holds per-request state) supports horizontal scaling; the orchestrator's in-process trace assembly is the main bottleneck for this.

Ecosystem evolution. Strava's announcement of an official MCP server (Strava, 2026) validates the architecture's bet on protocol-based integration and suggests that first-party MCP servers from data providers will progressively replace third-party API wrappers. As MCP adoption grows (Anthropic, 2024a), community-contributed servers (nutrition, air quality, social rides) could be added with a single URL. Enabling token-level SSE streaming and caching frequently repeated tool calls would further reduce the 8–15 second latency discussed in Section 8.3.

9.3 Closing Remarks

Training Copilot demonstrates that protocol-based data integration (MCP), multi-agent coordination (A2A), and domain-scoped specialist agents (LangGraph ReAct) can bridge the gap between fragmented fitness data and actionable training insight. The system runs in real time on commodity hardware and is designed to be extended by anyone who can write a Python function and a one-line configuration entry. As standardised agent protocols mature and wearable ecosystems become more interoperable, architectures like Training Copilot's—where intelligence is in the coordination layer, not in the data layer—offer a viable pattern for agentic applications across domains beyond sports analytics.

AI Usage Documentation

The following table documents all uses of generative AI tools during the development of this seminar paper and the Training Copilot software system. All AI-generated outputs were reviewed, verified, and adapted by the authors before inclusion.

Table 6 Documentation of AI tool usage throughout the project.

Scope	**AI Tool**	**Usage and Verification**
Writing (all sections)	DeepL	Translation of German drafts to English as a starting point; manually revised for academic tone.
Writing (all sections)	LanguageTool	Grammar and spelling corrections; accepted after review.
Writing (all sections)	Claude Code	Sentence-level rewriting, paragraph restructuring, and formulation improvements. All rephrased text reviewed against sources.
Literature review	Claude Projects	Paper discovery, screening, and summarisation of paper content. Each paper was read in full and all claims verified against the original PDF before citing. Verifications documented in a citation knowledge base.
LaTeX figures / tables	Claude Code	TikZ diagram generation, table formatting, and layout adjustments. Verified via PDF rendering.
Software development	Claude Code, GitHub Copilot	Code generation, debugging, architecture design, prompt engineering for specialist agents, evaluation framework and persona design, and synthetic test data generation. All code reviewed and tested before integration.

A Appendix

A.1 Code Listings

Listing 1 Server registry (simplified). Each URL is environment-overridable.

```
MCP_SERVERS = {
    "weather": _url("weather", 8101),
    "routes":  _url("routes", 8102),
    "strava":  _url("strava", 8103),
    "garmin":  _url("garmin", 8104),
    "calendar": _url("calendar", 8105),
}
```

Listing 2 Server implementation pattern (simplified).

```
mcp = FastMCP("weather", host="127.0.0.1",
              port=8101, stateless_http=True)

@mcp.tool()
def get_weather_forecast(lat: float, lon: float,
                         days: int = 7) -> dict:
    """Multi-day weather forecast for the given
    coordinates. Returns temperature, precipitation,
    wind, and UV index per day."""
    return _call_openmeteo(lat, lon, days)
```

A.2 MCP Server Tool Inventory

Table 7 provides the complete tool inventory across all MCP servers deployed in Training Copilot.

A.3 Evaluation Persona Details

Table 8 lists all ten evaluation personas with their types, goals, and expected focus areas.

A.4 Environment Configuration

Table 9 lists the key environment variables used by Training Copilot.

Table 7 Complete MCP tool inventory.

Server	Port	Tool Name	Description
Weather	8101	`get_current_weather`	Current conditions
		`get_weather_forecast`	Multi-day forecast
		`get_pollen_levels`	Pollen concentrations
		`get_uv_index`	UV index with category
Routes	8102	`geocode`	Place name to coordinates
		`plan_route`	A-to-B route
		`plan_circular_route`	Loop from start point
		`plan_park_loop`	Loop inside named area
		`explore_trails`	Trail search nearby
		`get_elevation_profile`	Elevation analysis
		`get_isochrone`	Reachability polygon
Strava	8103	`get_activities`	Recent activities
		`get_activity_stats`	Aggregate totals
		`get_athlete_profile`	Athlete profile + stats
		`get_training_trends`	Weekly volume trends
		`get_personal_bests`	Top performances
		`get_yearly_breakdown`	Year-over-year totals
		`get_gear_info`	Bike/shoe mileage
		`get_activity_detail`	Lap splits, HR, power
		`get_activity_streams`	Raw GPS streams
		`get_training_load`	CTL/ATL/TSB
		`delete_activity`	Remove an activity
		`analyze_performance_trends`	Multi-metric trend analysis
		`compare_activity_to_baseline`	Activity vs. personal baseline
Garmin	8104	`get_garmin_activities`	Activity list
		`get_garmin_activity_detail`	Per-lap splits
		`get_garmin_daily_health`	Daily wellness summary
		`get_garmin_heart_rate_timeline`	Intraday HR timeline
		`get_garmin_sleep`	Sleep stages + score
		`get_garmin_body_battery`	Energy metric timeline
		`get_garmin_hrv_status`	HRV + readiness
		`get_garmin_training_metrics`	VO_2max, load, status
		`get_garmin_wellness_trends`	Multi-day wellness trends
		`get_garmin_steps_timeline`	Intraday step counts
		`get_garmin_stress_timeline`	Intraday stress levels
		`get_garmin_body_composition`	Weight, BMI, body fat
		`get_activity_gps_track`	GPS track from activity
Calendar	8105	`list_calendars`	Available calendars
		`list_events`	Events in range
		`get_event`	Single event detail
		`create_event`	Create event
		`update_event`	Update event
		`delete_event`	Delete event

Table 8 Evaluation persona details.

Name	Type	Goal	Expected Focus
Julian	Triathlete	Recovery readiness for brick session	Recovery go/no-go
Priya	Triathlete	Training load trend + taper plan	CTL/ATL/TSB analysis
Marco	Triathlete	Post-workout zone + split review	HR zones, lap splits
Lena	Triathlete	Schedule training via weather + calendar	Time-window scheduling
Tom	Triathlete	Evidence-based technique advice	Cited literature
Sophie	Cyclist	Scenic loop with café stop	Planned scenic route
Ben	Cyclist	Weather go/no-go for ride	Weather assessment
Mara	Cyclist	Trail discovery + elevation	Trail options
Carlos	Cyclist	Year-over-year progress	Plain-language trends
Jana	Cyclist	Group loop for mixed abilities	Shareable route

Table 9 Key environment variables.

Variable	Default	Purpose
`LLM_PROVIDER`	`openai`	LLM backend (`openai`, `openai_official`, `gemini`)
`AGENT_MODEL`	—	Model for agent layer
`AGENT_LLM_MODEL`	—	Override model for agents
`OPENAI_BASE_URL`	—	KIT gateway URL
`ORS_API_KEY`	—	OpenRouteService key
`GARMIN_EMAIL`	—	Garmin Connect email
`CLIENT_ID`	—	Strava OAuth client ID
`MLFLOW_TRACKING_URI`	`:5001`	MLflow server URL
`ORCHESTRATOR_SPECIALISTS`	all	Enabled specialist agents

References

Acharya, D.B., Kuppan, K., Divya, B. (2025). Agentic AI: Autonomous intelligence for complex goals—a comprehensive survey. *IEEE Access*, *13*, 18912–18936.

10.1109/ACCESS.2025.3532853

Anthropic (2024a). *Introducing the model context protocol.* https://www.anthropic.com/news/model-context-protocol. (Blog post, November 25, 2024)

Anthropic (2024b). *Model context protocol specification.* https://modelcontextprotocol.io/specification/2025-11-25. (Open standard, version 2024-11-05)

Athletica.ai (2024). *Athletica: Adaptive endurance training plans powered by sports science.* https://athletica.ai/. (AI-driven adaptive training plans for runners, cyclists, and triathletes)

Barger, A.S. (2025). Artificial intelligence vs. human coaches: Examining the development of working alliance in a single session. *Frontiers in Psychology*, *15*, 1364054.

10.3389/fpsyg.2024.1364054

Bourdon, P.C., Cardinale, M., Murray, A., Gastin, P., Kellmann, M., Varley, M.C., ... Cable, N.T. (2017). Monitoring athlete training loads: Consensus statement. *International Journal of Sports Physiology and Performance*, *12*(Suppl 2), S2-161–S2-170.

10.1123/IJSPP.2017-0208

Dragoni, N., Giallorenzo, S., Lafuente, A.L., Mazzara, M., Montesi, F., Mustafin, R., Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. M. Mazzara & B. Meyer (Eds.), *Present and ulterior software engineering* (pp. 195–216). Springer. 10.1007/978-3-319-67425-4_12

Fuller, D., Colwell, E., Low, J., Orychock, K., Tobin, M.A., Simango, B., ... Taylor, N.G.A. (2020). Reliability and validity of commercially available wearable devices for measuring steps, energy expenditure, and heart rate: Systematic review. *JMIR mHealth and uHealth*, *8*(9), e18694.

10.2196/18694

Gabarron, E., Larbi, D., Rivera-Romero, O., Denecke, K. (2024). Human factors in AI-driven digital solutions for increasing physical activity: Scoping review. *JMIR Human Factors*, *11*, e55964.

10.2196/55964

Gabbett, T.J. (2016). The training–injury prevention paradox: should athletes be training smarter and harder? *British Journal of Sports Medicine*, *50*(5), 273–280.

10.1136/bjsports-2015-095788

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... Wang, H. (2024). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*. Retrieved from https://arxiv.org/abs/2312.10997

Google Cloud (2025). *Agent2agent (A2A) protocol specification.* https://github.com/a2aproject/A2A. (Announced April 9, 2025; donated to Linux Foundation June 2025)

Grand View Research (2026). *Sports analytics market size, share & trends analysis report, 2026–2033.* https://www.grandviewresearch.com/industry-analysis/sports-analytics-market. (Published June 2026. Market valued at USD 5.7B in 2025, projected USD 23.1B by 2033, CAGR 18.5%)

Grgic, J., Mikulic, P., Podnar, H., Pedisic, Z. (2017). Effects of linear and daily undulating periodized resistance training programs on measures of muscle hypertrophy: A systematic review and meta-analysis. *PeerJ*, *5*, e3695.

10.7717/peerj.3695

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., ... Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. *Proceedings of the thirty-third international joint conference on artificial intelligence (ijcai)*. Retrieved from https://arxiv.org/abs/2402.01680

Halson, S.L. (2014). Monitoring training load to understand fatigue in athletes. *Sports Medicine*, *44*(Suppl 2), S139–S147.

10.1007/s40279-014-0253-z

Hooren, B.V., Goudsmit, J., Restrepo, J., Vos, S. (2020). Real-time feedback by wearables in running: Current approaches, challenges and suggestions for improvements. *Journal of Sports Sciences*, *38*(2), 214–230.

10.1080/02640414.2019.1690960

IDC (2024). *Worldwide wearables market forecast, 2024–2028.* https://www.idc.com/promo/wearablevendor. (Global wearable device market report)

Impellizzeri, F.M., Woodcock, S., Coutts, A.J., Fanchini, M., McCall, A., Vigotsky, A.D. (2020). Training load and its role in injury prevention, part 2: Conceptual and methodologic pitfalls. *Journal of Athletic Training*, *55*(9), 893–901.

10.4085/1062-6050-501-19

Jayanti, M.A., & Han, X.Y. (2026). Enhancing model context protocol (MCP) with context-aware server collaboration. *arXiv preprint arXiv:2601.11595*. Retrieved from https://arxiv.org/abs/2601.11595

Jörke, M., Sapkota, S., Warkenthien, L., Vainio, N., Schmiedmayer, P., Brunskill, E., Landay, J.A. (2025). GPTCoach: Towards LLM-based physical activity coaching. *Proceedings of the chi conference on human factors in computing systems (chi)*. ACM. 10.1145/3706598.3713819

JSON-RPC Working Group (2013). *JSON-RPC 2.0 specification.* https://www.jsonrpc.org/specification. (Transport-agnostic remote procedure call protocol)

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... tau Yih, W. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 conference on empirical methods in natural language processing (emnlp)* (pp. 6769–6781). Retrieved from https://arxiv.org/abs/2004.04906

Kellmann, M., Bertollo, M., Bosquet, L., Brink, M., Coutts, A.J., Duffield, R., ... Beckmann, J. (2018). Recovery and performance in sport: Consensus statement. *International Journal of Sports Physiology and Performance*, *13*(2), 240–245.

10.1123/IJSPP.2017-0759

LangChain (2024). *LangGraph: Build resilient language agents as graphs.* https://docs.langchain.com/oss/python/langgraph/overview. (Framework for building stateful, multi-actor LLM applications)

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in neural information processing systems (neurips)* (Vol. 33, pp. 9459–9474). Retrieved from https://arxiv.org/abs/2005.11401

Lundstrom, C.J., Foreman, N.A., Biltz, G. (2023). Practices and applications of heart rate variability monitoring in endurance athletes. *International Journal of Sports Medicine*, *44*, 9–19.

10.1055/a-1864-9726

Luo, T.C., Aguilera, A., Lyles, C.R., Figueroa, C.A. (2021). Promoting physical activity through conversational agents: Mixed methods systematic review. *Journal of Medical Internet Research*, *23*(9), e25486.

10.2196/25486

McKinsey & Company (2023). *The economic potential of generative AI: The next productivity frontier.* https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier. (Estimates USD 2.6–4.4 trillion annual value across industries)

MLflow (2026). *Multi-turn conversation evaluation with MLflow.* https://mlflow.org/blog/multiturn-evaluation/. (Tutorial on automated multi-turn agent evaluation)

Monteiro-Guerra, F., Rivera-Romero, O., Fernandez-Luque, L., Caulfield, B. (2020). Personalization in real-time physical activity coaching using mobile applications: A scoping review. *IEEE Journal of Biomedical and Health Informatics*, *24*(6), 1738–1751.

10.1109/JBHI.2019.2947243

Morton, R.H., Fitz-Clarke, J.R., Banister, E.W. (1990). Modeling human performance in running. *Journal of Applied Physiology*, *69*(3), 1171–1177.

10.1152/jappl.1990.69.3.1171

OpenAI (2023). *GPT-4 technical report.* Retrieved from https://arxiv.org/abs/2303.08774

Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th annual acm symposium on user interface software and technology (uist).* ACM. 10.1145/3586183.3606763

Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E. (2023). Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*. Retrieved from https://arxiv.org/abs/2305.15334

Plews, D.J., Laursen, P.B., Stanley, A.E., Buchheit, M. (2013). Training adaptation and heart rate variability in elite endurance athletes: Opening the door to effective monitoring. *Sports Medicine*, *43*(9), 773–781.

10.1007/s40279-013-0071-8

Puce, L., Bragazzi, N.L., Currà, A., Trompetto, C. (2025). Harnessing generative artificial intelligence for exercise and training prescription: Applications and implications in sports and physical activity—a systematic literature review. *Applied Sciences*, *15*(7), 3497.

10.3390/app15073497

Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., ... Wen, J.-R. (2025). Tool learning with large language models: A survey. *Frontiers of Computer Science*, *19*(8), 198343.

10.1007/s11704-024-40678-2

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (emnlp-ijcnlp)* (pp. 3982–3992). Retrieved from https://aclanthology.org/D19-1410/

Rothschild, J.A., Stewart, T., Kilding, A.E., Plews, D.J. (2024). Predicting daily recovery during long-term endurance training using machine learning analysis. *European Journal of Applied Physiology*, *124*, 3279–3290.

10.1007/s00421-024-05530-2

Sahoo, P., Singh, A.K., Saha, S., Jain, V., Mondal, S., Chadha, A. (2024). A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*. Retrieved from https://arxiv.org/abs/2402.07927

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., ... Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems (neurips)* (Vol. 36). Retrieved from https://arxiv.org/abs/2302.04761

Seckin, A.C., Ates, B., Seckin, M. (2023). Review on wearable technology in sports: Concepts, challenges and opportunities. *Applied Sciences*, *13*(18), 10399.

10.3390/app131810399

Shankar, S., Zamfirescu-Pereira, J.D., Hartmann, B., Parameswaran, A.G., Arawjo, I. (2024). Who validates the validators? Aligning LLM-Assisted evaluation of LLM outputs with human preferences. *Proceedings of the 37th annual acm symposium on user interface software and technology (uist)*. ACM. 10.1145/3654777.3676450

Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y. (2023). HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. *Advances in neural information processing systems (neurips)* (Vol. 36). Retrieved from https://arxiv.org/abs/2303.17580

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems (neurips)* (Vol. 36). Retrieved from https://arxiv.org/abs/2303.11366

Statista (2025). *Fitness apps: Number of downloads in Germany 2017–2025*. https://de.statista.com/prognosen/1424687/fitness-apps-anzahl-downloads/. (Downloads rising from 41.37M (2017) to 135.22M (2025))

Strava (2025). *Year in sport trend report 2025*. https://contentful-assets.strava.com/Strava-Year-In-Sport-Trend-Report-2025-US.pdf. (30% of Gen Z plan higher fitness spending for 2026; 46% would use AI for sports analysis)

Strava (2026). *An update to our developer program.* https://communityhub.strava.com/insider-journal-9/an-update-to-our-developer-program-13428. (Strava Developer Hub blog post, June 2026. Introduces paid API tiers and announces Strava MCP)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., ... Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems (neurips)* (Vol. 30). Retrieved from https://arxiv.org/abs/1706.03762

Vemuri, A., Decker, K., Saponaro, M., Dominick, G. (2021). Multi agent architecture for automated health coaching. *Journal of Medical Systems*, *45*(11), 95.

10.1007/s10916-021-01771-2

Venter, Z., Gundersen, V., Scott, D.J., Barton, D.N. (2023). Bias and precision of crowdsourced recreational activity data from Strava. *Landscape and Urban Planning*, *232*, 104686.

10.1016/j.landurbplan.2023.104686

Wackerhage, H., & Schoenfeld, B.J. (2021). Personalized, evidence-informed training plans and exercise prescriptions for performance, fitness and health. *Sports Medicine*, *51*, 1805–1813.

10.1007/s40279-021-01495-w

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., ... Wen, J.-R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, *18*(6), 186345.

10.1007/s11704-024-40231-1

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems (neurips)* (Vol. 35). Retrieved from https://arxiv.org/abs/2201.11903

WHOOP (2024). *WHOOP coach: AI performance coaching powered by OpenAI.* https://openai.com/index/whoop/. (In-app AI coach using biometric data for natural-language health guidance)

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., ... Wang, C. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *Proceedings of the iclr workshop on llm agents.* Retrieved from https://arxiv.org/abs/2308.08155

Yang, Y., Chai, W., Song, Y., Qi, H., Wen, C., Li, Y., ... Zhang, P. (2025). A survey of AI agent protocols. *arXiv preprint arXiv:2504.16736*. Retrieved from https://arxiv.org/abs/2504.16736

Yao, S., Shinn, N., Razavi, P., Narasimhan, K. (2024). τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*. Retrieved from https://arxiv.org/abs/2406.12045

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *Proceedings of the eleventh international conference on learning representations (iclr).* Retrieved from https://arxiv.org/abs/2210.03629

This template is based on the LaTeX template of Springer Nature. The original template is available at https://www.springernature.com/gp/authors/campaigns/latex-author-support.